# The mini coding agent: course manual

### A staged theory companion to `PLAN.md`

This manual explains *why* the code in each stage looks the way it does. The build plan tells you what to create and how to prove it works; this manual gives you the mental model, the design vocabulary, and the failure modes so that each stage is a concept you own rather than a file you copied.

Read one stage of the plan, then read the matching chapter here before you start typing. When you finish the stage, answer the checkpoint questions at the end of the chapter, then open each question's dropdown to compare your reasoning. If you can explain the answers without looking, you are ready to build the next layer.

> **Source material.** The long-form explanation of the finished harness is in `../mini-coding-agent-js/explainer-harness-js.md`. This manual reorganises that theory around the 16 stages in `PLAN.md` and adds teaching notes, checkpoints and common mistakes. The code excerpts here are illustrative; the plan is authoritative for exact signatures and acceptance criteria.

---

# Part 0: The idea before the code

CONTENTS

# What is in this manual


0.1 What an agent is 3

0.2 The six-line agent 4

0.3 Why we build conversation first 5

0.4 The architecture you are growing 6

0.5 The ten principles of the harness 8

Stage 0: Scaffold 9

Stage 1: Terminal language ( ui.js ) 11

Stage 2: Config and the model client 14

Stage 3: The simplest chat loop 20

Stage 4: Conversation history 23

Stage 5: The system prompt 26

Stage 6: Path policy 29

Stage 7: Read-only file tools 32

Stage 8: Sandbox and write tools 37

Stage 9: Command execution 41

Stage 10: Long-term memory 46

Stage 11: Lifecycle hooks 51

Stage 12: The ReAct loop 56

Stage 13: Offline demo 60

Stage 14: Sessions, trimming and CLI polish 63

Stage 15: The full test suite 67

Stage 16: Documentation and final acceptance 71

A. The message protocol 74

B. The safety model in one page 75

C. Every bound and cap 76

D. Debugging by symptom 77

E. Glossary 78

F. Reading the finished code 79

Closing note 80


Page numbers refer to the folio at the foot of each page.

# 0.1 What an agent is

Strip the branding and the product categories away from any coding agent and three things remain:

1. **A model** that can only produce text. It cannot read a file, run a test, or remember yesterday. It maps a conversation to the next message.
2. **Tools** that can actually do something: list a folder, read a file, write a file, run a command.
3. **A loop** that takes the model's text, notices when it asks for a tool, runs the tool, and hands the result back to the model as another message.

Everything else you have seen a coding agent do — planning, sub-agents, retries, permissions, skills, project rules, memory — is a layer on top of those three. The layers matter, but they only make sense once the core is visible. This course builds the core in public, then adds two layers that earn their place immediately: **memory** and **lifecycle hooks**.

A chatbot is the same program with the loop deleted:

- one model call,
- one answer,
- no way to look at the world,
- no way to react to what it saw.

The difference between "answers questions about your code" and "changes your code" is not a bigger model. It is the loop. That sentence is worth memorising before Stage 0.

## The three-column mental model

| PIECE | LIVES IN | SPEAKS | CAN CHANGE THE WORLD? |
|---|---|---|---|
| Model | OpenRouter | Text and tool-call JSON | No |
| Tools | `src/tools.js` | Strings in, strings out | Yes, through the sandbox |
| Loop | `src/agent.js` (from Stage 12) | Messages | No, but it decides when tools run |

The model never touches the filesystem. It *names* a tool and supplies arguments. The harness validates those arguments, enforces policy, executes the handler, and returns a string. That separation is the entire security architecture, and it is why a tool call can be refused without crashing anything.

# 0.2 The six-line agent

Here is the whole idea, written as pseudocode:

```
messages = [system prompt, user task]

repeat up to maxSteps:
    reply = model(messages, tools)          # text in, text out
    if reply has no tool calls: return reply.content
    for each tool call:
        messages.append(tool result)        # observation
```

This is not a simplification of an agent. It *is* the agent. Every stage in this course adds plumbing around one of those lines:

- Stages 0–5 make the first `model(messages)` call possible, and make the message array meaningful.
- Stages 6–9 make the `tool result` line real, with safety around it.
- Stage 10 makes memory cross process boundaries.
- Stage 11 makes the loop observable and extensible.
- Stage 12 extracts the loop into a library that can be tested.
- Stages 13–16 prove the behaviour, polish the program, and document it.

### Think, act, observe, repeat

When you watch the finished harness run, every step reads like a sentence:

```
── step 1/12 ──────────────────────────────────────
think: Creating a script, then running it to check it works.
tool → create_file {"path":"demo/hello.js","content":"..."}
result ← created demo/hello.js (72 bytes)
```

- The model **thought**: it produced text explaining its intention.
- It **acted**: it emitted a tool call.
- The world **answered**: the harness returned a string.
- The loop went round again with that answer in hand.

That is the ReAct pattern: **Re**ason, **Act**, observe. You do not need a framework to implement it. You need an array, a `for` loop, and a `fetch`.

> **Key idea.** The loop is not an implementation detail. It is the agent. Delete the `for` and you have a chatbot with tools it will never call.

# 0.3 Why we build conversation first

The build order in `PLAN.md` is deliberate and slightly unusual. A traditional engineering plan would build the sandbox, then the tools, then the model client, and only then let the program talk. That plan is easier to diagram and much harder to learn from, because nothing visible happens until late.

This course builds in the opposite direction:

1. **Make it talk.** Config, HTTP, one model call. You have a chatbot in Stage 3.
2. **Give it memory of the conversation.** Stage 4.
3. **Give it a job description.** Stage 5.
4. **Give it hands.** Stages 6–9, one capability at a time.
5. **Give it persistence and observability.** Stages 10–11.
6. **Formalise the loop.** Stage 12.
7. **Prove it.** Stages 13–16.

The reason is motivational and pedagogical: every stage produces a runnable program that is immediately more capable than the last. You can feel the feature. You also learn the failure modes in the order a real user meets them:

- A bad key is a Stage 2 problem.
- A model that forgets the previous turn is a Stage 4 problem.
- A model that writes the wrong file is a Stage 8 problem.
- A command that hangs is a Stage 9 problem.
- A model that loops forever is a Stage 12 problem.

Building the loop first would hide all of those behind a harness you do not yet understand.

> **Rule of the course.** Do not skip ahead. Each stage adds one concrete capability and one conceptual boundary. If the program does not do something new and useful after a stage, the stage is not finished.

# 0.4 The architecture you are growing

By the end, the import graph points in one direction:

```
agent.js ── THE LOOP ──────────────────┐
   ├── prompt.js     the rules + memory
   ├── heartbeat.js  still waiting     │  what one step
   ├── llm.js        one POST          │  needs
   └── tools.js      schemas + run
         ├── workspace.js  containment
         └── sandbox.js    write fence
                                       │
cli.js ── THE PROGRAM ─────────────────┘  everything a
   ├── agent.js      runTask             person needs to
   ├── config.js     .env + defaults     actually run it
   ├── memory.js     src/memory.md
   ├── demo.js       the fake model
   ├── hooks.js      named moments
   └── ui.js         every chalk call
```

The important line is the asymmetry between the halves:

- `agent.js` **is a library.** It knows nothing about flags, colours, `.env`, `process.exit`, or the terminal. It takes dependencies as parameters and returns a plain object.
- `cli.js` **is the program.** It reads flags, loads config, prints banners, builds dependencies, calls `runTask`, and turns the result into an exit code.

Before Stage 12, `cli.js` owns the conversation loop directly. We extract it only once it is complex enough to deserve its own file. That order is intentional: extracting a loop you have not yet written by hand produces a library nobody understands.

### The file-by-file questions

Each file answers one question a reader will ask. Keep this table nearby as you build; it is the map of the course.

| FILE | THE QUESTION IT ANSWERS | INTRODUCED |
|---|---|---|
| `src/ui.js` | Where is the colour? (Nowhere else.) | Stage 1 |
| `src/config.js` | Which key, model and limits — and where did each value come from? | Stage 2 |
| `src/llm.js` | What exactly goes over the wire? | Stage 2 |
| `src/cli.js` | What does a person type? | Stage 3 |
| `src/prompt.js` | What is the model told before it starts? | Stage 5 |
| `src/workspace.js` | How is a path stopped from leaving the sandbox? | Stage 6 |
| `src/tools.js` | What can the model do, and what happens when it asks? | Stage 7 |
| `src/sandbox.js` | May the agent change this file at all? | Stage 8 |
| `src/memory.js` | What does it know that survives the process? | Stage 10 |
| `src/hooks.js` | Where can outside code get a turn, and what may it change? | Stage 11 |
| `src/agent.js` | Where is the loop? | Stage 12 |

| FILE | THE QUESTION IT ANSWERS | INTRODUCED |
|---|---|---|
| `src/heartbeat.js` | Why doesn't a 60-second request look like a crash? | Stage 12 |
| `src/demo.js` | What runs when there is no network, no key and no model? | Stage 13 |

## 0.5 The ten principles of the harness

These principles are the through-line of the whole course. Each stage implements one or more of them. They are also the vocabulary to use when you explain your code to somebody else.

| # | PRINCIPLE | WHERE YOU WILL SEE IT |
|---|---|---|
| 1 | **Tool calling is the model's only ability to act.** The schemas *are* the prompt. | `tools.js`, Stage 7 |
| 2 | **The loop is the agent.** | `agent.js`, Stage 12 |
| 3 | **Everything is messages.** One array, four roles, no hidden state. | Stage 4 onward |
| 4 | **Errors are observations, not crashes.** A failed tool returns text. | Stage 7 onward |
| 5 | **Exit codes close the loop.** `exit code 0` versus `1` is how the model learns whether it worked. | Stage 9 |
| 6 | **Human in the loop.** Commands ask by default; hard denies cannot be overridden. | Stage 9 |
| 7 | **Least privilege.** Reads are open, writes are fenced, child processes get a minimal environment. | Stages 6–9 |
| 8 | **Bounded autonomy.** `maxSteps`, timeouts, truncation guard, memory caps. | Stages 2, 9, 10, 12 |
| 9 | **Extension points belong at named moments.** Hooks, not edits to the loop. | Stage 11 |
| 10 | **Memory is a file, not a mystery.** | Stage 10 |

# Phase A: A chatbot that talks

**Goal of the phase:** a working OpenRouter chat client. No tools, no sandbox, no memory — just you and the model, with configuration you can trust and output you can read.

**What you can do at the end:** hold a multi-turn conversation in the terminal, with a system prompt that shapes the model's behaviour.

# Stage 0: Scaffold

## What this stage is really about

Stage 0 looks like ceremony: a `package.json`, a `.gitignore`, two empty folders. It is actually where you decide two design constraints that the rest of the course depends on.

**Constraint 1: one runtime dependency.** The project uses `chalk` for colour and nothing else. No web framework, no agent SDK, no database, no ORM. The point is not asceticism for its own sake. Every dependency you add is a layer between you and the behaviour you are trying to understand. When the model call is a `fetch` you can read, a timeout is an `AbortSignal` you can trace. When it is an SDK, a timeout is a configuration option whose implementation you cannot see.

**Constraint 2: a workspace folder.** The agent operates from the repository root but may only *write* inside `workspace/`. This is the containment rule that Stage 6 formalises and Stage 8 enforces. Even at Stage 0, the folder exists so that the rule has somewhere to point.

## The layout and why it is flat

```
mini-coding-agent-js/
  package.json          type: module, one dependency
  .gitignore            node_modules, .env, workspace/*
  src/                  all the code
  test/                 all the tests
  workspace/            the only folder the agent may write to
  .env                  your key (never committed)
```

The `src/` folder is flat on purpose. There is no `src/core/`, `src/adapters/`, `src/domain/`. Thirteen files that import in one direction do not need a directory hierarchy, and a flat folder makes the import graph visible in a single listing. Architecture is not the same thing as folder depth.

ES modules (`"type": "module"`) are used throughout because:

- `import.meta.dirname` gives a reliable project root without `__dirname` workarounds.
- Top-level `await` is available.
- `node --test` and modern `fetch` fit naturally.
- The browser and Node module semantics are closer, which matters if you later move code.

## The workspace rule

All files the agent creates or edits go in `workspace/`. The agent starts commands there. The sandbox root is there. But the agent may *read* outside it — a library, a log, a `package.json` one folder up. That read/write asymmetry is introduced conceptually here and becomes code in Stage 6.

Two details to internalise now:

- The agent does not create subfolders of `workspace/` unless a tool call asks for a path with a subfolder, or a write handler creates parents.
- Relative paths in tool calls resolve from `workspace/`, not from wherever you launched the process.

## Why `.gitignore` matters in a course

The agent will write files. Some of them will be experiments, some will be wrong, and some will contain output from commands. Ignoring `workspace/*` (except `.gitkeep`) keeps your repository about the harness, not about the demo debris. Ignoring `.env` is not optional: an API key in git history is a leaked key.

## Checkpoint

**1. Why does the course insist on a single runtime dependency?**

A. Every dependency is a layer between you and the behaviour; one dependency keeps the model call, timeout and filesystem visible.

**2. What is the difference between where the process starts and where the agent may write?**

A. The process starts wherever you launch it; the agent may only write inside `workspace/`.

**3. Why is `workspace/.gitkeep` committed while the rest of `workspace/` is ignored?**

A. `.gitkeep` preserves the empty folder in git; everything else in `workspace/` is generated and should not be committed.

---

# Stage 1: Terminal language (`ui.js`)

## What this stage is really about

Stage 1 is not "make it pretty". It is **separation of presentation from logic**, and it teaches a contract that survives all the way to the test suite: *every visual helper returns a string; the caller decides whether to print it.*

That contract has three consequences:

1. `agent.js` can accept an injected `log` function and be tested without a terminal.
2. The entire palette can be changed in one file.
3. Model output containing backslashes, `<red>`, or ANSI-looking text prints verbatim, because `chalk` styles the exact string you give it rather than parsing markup.

A console library that interprets inline tags would quietly corrupt the very output the user is reading. A model that writes a file containing `<red>` should show `<red>`, not a colour change.

## The return-a-string rule

```
// good: pure presentation
export const ui = {
  title: (text) => chalk.bold.cyan(text),
  step: (step, max, width = 40) =>
    chalk.blue(`── step ${step}/${max} ${'─'.repeat(width)}`),
  think: (text) => `${chalk.cyan('think:')} ${text}`,
};

// caller decides when and where
log(ui.step(1, 12));
log(ui.think('Creating a script, then running it.'));
```

Nothing inside `ui.js` calls `console.log`. If a helper printed directly, the loop would be untestable without capturing stdout, and the CLI could not redirect output.

## Result classification: the colour is derived, not reported

A tool does not say "this failed". It returns a string, and the UI infers failure:

```
const isFailure = (text) =>
  /^(ERROR|DENIED)\b/.test(text) ||
  /\bexit code (?!0\b)\d+/.test(text);
```

That small regex is a visual contract for the whole course:

- **red** means the model has a problem to solve on the next step;
- **green** means the observation is clean.

It also explains why `ui.result` prints the first line prominently and puts extra detail underneath. A `run_command` result may be many lines; the first line is the verdict, the rest is evidence.

## `explainExit` and the teaching value of exit codes

Exit code `1` is not a number the model can interpret by itself. `explainExit(1)` turns it into plain English: the command ran and then reported an error; the cause is in stderr; fix that, do not just re-run.

This is a small piece of prompt engineering in the UI layer, and it matters more than it looks. A model that sees only `exit code 1` may retry the same command. A model that sees *why* the code means failure is more likely to read stderr and change approach.

`explainExit` covers the common families:

- `0` success;
- `1` general failure;
- `2` shell misuse or bad argument;
- `3` a command-specific failure (not universal);
- `126` found but not executable;
- `127` command not found;
- `128–164` terminated by a signal, computed as `128 + signal number`;
- Windows-specific codes such as `9009` (command not found in `cmd.exe`).

You do not need every mapping memorised. You need the design idea: **turn machine signals into human explanations the model can act on.**

## `forbidden` must be loud and greppable

```
⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt
```

Bold red capitals, one line, the action and the requested path. Why so loud?

- It is the one line an audience should notice.
- It is greppable in logs.
- It is distinguishable from an ordinary `ERROR`.
- It is printed by exactly one place in the codebase (`sandbox.forbid`), so there is no ambiguity about where a refusal came from.

## Model output verbatim

`chalk` does not parse tags. If a tool result contains `\n`, `\t`, `C:\Users\...`, or JSON with quotes, it prints as-is. Do not "helpfully" strip or interpret model output. The model needs to see the exact characters, and so do you.

## Checkpoint

**1. Why should `ui.js` never call `console.log`?**

A. So the caller controls printing, the loop can be tested with an injected `log`, and output can be redirected.

**2. How does `ui.result` decide between green and red?**

A. `isFailure` checks the first line for `ERROR` / `DENIED` or a non-zero `exit code`.

**3. Why is `FORBIDDEN` deliberately different from `ERROR`?**

A. `FORBIDDEN` is a policy refusal, not a tool failure; it must be loud, greppable and emitted from one place.

---

# Stage 2: Config and the model client

This stage has two halves with two different jobs: **configuration** decides what to send and where the values came from; **the client** sends it and turns transport failures into errors the program can handle.

## 2a. Configuration: precedence as a teaching tool

Three sources compete for the API key and the model name:

```
--model flag          ← highest
.env (nearest file)   ← beats the shell, on purpose
exported environment  ← lowest for anything .env defines
```

The unusual part is `.env` beating the shell. Most tools do the opposite. This course deliberately inverts it because a stale `OPENROUTER_API_KEY` in `~/.bashrc` would otherwise silently outrank the key you just wrote, and you would spend an afternoon debugging a `401`. Variables that `.env` does not mention still come from the environment.

**Why report provenance?** The banner says `model: ... (from .env)` or `key: sk-or...aaa5 (73 chars) from shell`. When a demo goes wrong, the first question is always "which value did it actually use?" A configuration system that cannot answer that question is a configuration system that wastes your time.

## Finding `.env`: nearest file wins

```
export function findEnvFile(startDir) {
  let dir = path.resolve(startDir);
  for (;;) {
    const candidate = path.join(dir, '.env');
    if (fs.existsSync(candidate) && fs.statSync(candidate).isFile()) return candidate;
    const parent = path.dirname(dir);
    if (parent === dir) return null; // filesystem root
    dir = parent;
  }
}
```

Walking upward from the project root means a single `.env` at the repository root works for a package one level down. Missing `.env` is not an error. You may be running with a shell key, or with `--demo`, which needs no key at all.

**Parsing** `.env` supports the forms you actually meet in the wild:

- `KEY=VALUE`;
- optional `export` prefix;
- `#` comments;
- quoted values, including spaces;
- blank values, which are ignored.

Do not use a full dotenv library for this. The parser is part of the lesson: configuration is text, and text you can read is configuration you can debug.

### Fingerprinting instead of printing

```
fingerprint('sk-or-v1-abcdefghijklmnopqrstuvwxyz1234')
// => 'sk-or...1234 (37 chars)'
```

Rules:

- unset → `not set`;
- length ≤ 12 → all `*`;
- otherwise → first 5 characters, `...`, last 4, plus the length.

A fingerprint is enough to tell two keys apart on stage and useless in a screenshot. Never print the whole key, even in "debug" mode. The habit is the point.

### Model normalisation: making a helpful guess visible

OpenRouter requires `vendor/model`. A `.env` carried over from another project usually contains a bare name such as `gpt-5` or `claude-sonnet-4.5`. Rather than failing, the config maps prefixes:

| PREFIX | VENDOR |
|---|---|
| `deepseek` | `deepseek` |
| `claude` | `anthropic` |
| `gpt`, `o1`, `o3`, `o4`, `chatgpt` | `openai` |
| `gemini` | `google` |
| `grok` | `x-ai` |
| `llama` | `meta-llama` |
| `qwen` | `qwen` |
| `mistral`, `mixtral` | `mistralai` |

Two rules keep the magic honest:

1. If the name already contains `/`, it is left alone.
2. If normalisation changed the value, the banner says so in a dim note.

A helpful guess that is invisible is a future bug. A helpful guess that announces itself is a convenience.

### `loadConfig` returns more than values

The config object should carry both the resolved values and their sources:

```
{
  apiKey, apiKeySource,        // 'env' | 'shell' | 'unset'
  model, modelSource,          // 'flag' | 'env' | 'default' | 'normalised'
  envFile,                     // path or null
  commandPolicy,               // 'ask' | 'allow' | 'deny'
  maxSteps,
  requestTimeoutMs,
  workspaceRoot,
  memoryFile,
  memoryMaxEntries,
  memoryPromptChars,
}
```

The CLI does not have to reconstruct any of this. The library returns facts, and the program prints them.

## Defaults as a design statement

| SETTING | DEFAULT | WHY |
|---|---|---|
| Model | `deepseek/deepseek-v4.1-flash-1017` | Cheap, capable, available; the course is not a model review |
| `CMD_POLICY` | `ask` | Safe by default; the human is the last line of defence |
| `MAX_STEPS` | `12` | Enough to look, edit, run, and recover; small enough to bound cost |
| `REQUEST_TIMEOUT_MS` | `90000` | Slow providers exist; a hung socket does not |
| `WORKSPACE_DIR` | `workspace` | Containment has a name |
| `MEMORY_FILE` | `src/memory.md` | Beside the code, outside the sandbox |
| `MEMORY_MAX_ENTRIES` | `60` | The file cannot grow without limit |
| `MEMORY_PROMPT_CHARS` | `1200` | The injected digest cannot eat the context window |

Defaults are not arbitrary. Every one of them is a bound or a safety posture, and later chapters explain the mechanism behind each.

## 2b. The model client: one POST

There is no SDK in this project, on purpose. A tool-calling request to an OpenAI-compatible endpoint is a `POST` with three interesting fields: `model`, `messages`, `tools`. The whole client is roughly 66 lines.

```
export async function chat({ apiKey, model, messages, tools, signal, timeoutMs = 90_000 }) {
  if (!apiKey) throw new Error('OPENROUTER_API_KEY is not set');

  const timeout = AbortSignal.timeout(timeoutMs);
  const abort = signal ? AbortSignal.any([signal, timeout]) : timeout;

  const res = await fetch('https://openrouter.ai/api/v1/chat/completions', {
    method: 'POST',
    signal: abort,
    headers: {
      Authorization: `Bearer ${apiKey}`,
      'Content-Type': 'application/json',
      'X-Title': 'min-coding-js',
    },
    body: JSON.stringify({ model, messages, tools, max_tokens: 2048 }),
  });

  const text = await res.text();
  if (!res.ok) {
    const hint = res.status === 401 ? ' (bad or revoked key)'
      : res.status === 402 ? ' (out of credits)' : '';
    throw new Error(`OpenRouter ${res.status}${hint}: ${text.slice(0, 400)}`);
  }
  return JSON.parse(text);
}
```

## The two timeout layers

`AbortSignal.timeout(timeoutMs)` is the entire liveness story at the transport layer. A socket that never answers becomes an error after 90 seconds instead of a hung agent.

`AbortSignal.any([signal, timeout])` composes two reasons to stop:

- the caller's cancellation signal (for Ctrl-C or tests), and
- the timeout.

`AbortSignal.any` is why Node 20 is a prerequisite. Without it you would have to wire up event listeners and remember to remove them.

The heartbeat in Stage 12 covers the same worry at the human layer: `… still waiting for the model (25s)`. One is about correctness; the other is about perception. You need both.

## Failures here throw; failures in tools do not

This is the most important distinction in the stage:

- A bad key, a `402`, a dropped connection, a non-JSON body: the model cannot fix these. The run stops, the CLI exits `1`, and the human fixes the configuration.
- A failed edit, a missing file, a denied command: the model *can* fix these. They come back as text observations and the loop continues.

Ask yourself at every error site: **can the model do anything about this?** If yes, return a string. If no, throw.

## Error handling details worth copying

- **Read the body as text first, then parse.** `res.json()` throws an unhelpful error when the body is HTML from a proxy. `res.text()` lets you include a truncated preview in the error.
- **Truncate the preview** (400 characters). Error bodies can be large and can contain the request.
- **Name the status and the likely cause.** `401 (bad or revoked key)` and `402 (out of credits)` save a support round-trip.
- **Handle a `.error` field in an otherwise-200 response.** Some providers return an error object with HTTP 200. Check for it.
- **Timeout message is explicit:** `request timed out after <seconds>s`. Do not surface `AbortError` unchanged.

## `sanitizeAssistant`: do not echo the scratchpad

Providers return extra fields on the assistant message: `reasoning`, `reasoning_content`, refusal blocks, annotations. Echoing them back re-sends the model's private thinking on every subsequent step — tokens you pay for, and a context that fills with stale reasoning.

```
export function sanitizeAssistant(message) {
  const out = { role: 'assistant', content: message.content ?? '' };
  if (Array.isArray(message.tool_calls) && message.tool_calls.length > 0) {
    out.tool_calls = message.tool_calls;
    if (message.content == null) delete out.content;
  }
  return out;
}
```

Two subtleties:

1. **Keep only `role`, `content`, `tool_calls`.** Everything else is provider-specific.
2. **Delete `content` when it is null and there are tool calls.** An empty string is a valid content field; `null` alongside tool calls is not, for some providers. The conditional delete keeps the message valid without inventing a value.

## Why `max_tokens` is small

`max_tokens: 2048` is deliberately small: it keeps each step cheap and keeps whole files out of a single reply. The cost is that long answers get cut off — which is exactly what the truncation guard in Stage 12 is for. A small limit plus a guard is better than a large limit plus a hung context window.

## Checkpoint

**1. Why does `.env` beat an exported shell variable in this project?**

A. A stale shell key must not silently outrank the project's key; `.env` is the project's explicit choice.

**2. What does `fingerprint` protect against, and why is a length included?**

A. It lets you distinguish keys without exposing them; the length helps identify a truncated or padded value.

**3. Why does `chat()` throw on a `401` but `runTool()` return an `ERROR` string for a bad edit?**

A. A `401` is not fixable by the model; a bad edit is.

**4. What breaks if you echo `reasoning_content` back to the provider?**

A. It re-sends private reasoning, costs tokens, and fills context with stale thinking.

**5. What happens when the caller passes a signal and the timeout fires first?**

A. `AbortSignal.any` aborts on whichever fires first; the error should explain which.

---

# Stage 3: The simplest chat loop

## What this stage is really about

Stage 3 is the first time the program says something useful. It is also the first time you wire three layers together:

- **config** (`config.js`) supplies the key, model and timeout;
- **transport** (`llm.js`) sends one request;
- **presentation** (`ui.js`) prints the answer.

The goal is immediate gratification. Resist the urge to add tools, memory, hooks, or the full loop. This stage is one task, one model call, one answer.

## The shape of the program

```
async function main() {
  const opts = parseArgs(process.argv.slice(2));
  if (opts.help) return usage();

  const config = loadConfig({ model: opts.model });
  const task = opts.task ?? await askTask();

  if (!task) {
    console.error('no task given');
    process.exitCode = 2;
    return;
  }

  const messages = [
    { role: 'system', content: 'You are a helpful assistant.' },
    { role: 'user', content: task },
  ];

  const reply = await chat({
    apiKey: config.apiKey,
    model: config.model,
    timeoutMs: config.requestTimeoutMs,
    messages,
  });

  console.log(ui.answer(reply.choices?.[0]?.message?.content ?? ''));
}
```

Everything here is deliberately simple. The interesting teaching points are in the edges.

## Reading user input with `readline`

`askTask({ input, output })` is a small readline loop:

- Prompt: `task ›`, with the label green and the arrow white.
- Trim the raw input.
- Blank lines re-ask rather than submitting an empty task.
- Ctrl-C returns `null` (do not crash with an unhandled `SIGINT`).

- Use `for await (const raw of rl)` rather than a single `question()` call, so multi-line chunks and pasted input behave predictably.

The last point is a real-world detail. A model may not care, but a human pasting a multi-line task does. The prompt loop is part of the UX, not an afterthought.

## Flag parsing without a library

```
function parseArgs(argv) {
  const opts = {};
  const rest = [];
  for (let i = 0; i < argv.length; i += 1) {
    const arg = argv[i];
    if (arg === '--help' || arg === '-h') opts.help = true;
    else if (arg === '--model') { i += 1; opts.model = argv[i] ?? ''; }
    else if (arg === '--max-steps') { i += 1; opts.maxSteps = Number(argv[i]); }
    else rest.push(arg);
  }
  return { opts, task: rest.join(' ') || undefined };
}
```

Why hand-rolled?

- The flag set is small and stable.
- Every parser library is another dependency and another failure mode.
- A course should show you the shape of argument parsing, not hide it.

Two rules to preserve:

1. `--help` must work with no API key. It is documentation, not a network operation.
2. Everything that is not a flag becomes the task string. That is why `node src/cli.js say hello` does the obvious thing.

## Exit codes are part of the interface

Even at Stage 3, decide what failure means:

- **0**: a reply was printed.
- **1**: the request failed (network, key, credits, timeout).
- **2**: no task and stdin is not a terminal.

`2` deserves explanation. If you run the CLI in a CI job with no stdin, the prompt cannot be answered. Printing a hint and exiting `2` is better than hanging forever waiting for input. It also makes the program scriptable: a non-TTY caller learns immediately that it must pass a task argument.

## The "one call" limitation

After Stage 3 the program has no memory. If you ask "what did I just say?", the model cannot know, because the `messages` array was created fresh for this request and thrown away afterwards. That is not a model failure. It is an architectural fact: **the conversation is the array, and the array is local to the call.** Stage 4 fixes it.

## Checkpoint

**1. Why does `--help` need to work without a key?**

A. `--help` is documentation; requiring a key to read usage is hostile.

**2. Why do we return `null` on Ctrl-C instead of letting the process throw?**

A. Ctrl-C is a normal user action, not an exception; returning `null` lets the loop exit cleanly.

**3. What does exit code `2` mean, and why is it different from `1`?**

A. `2` means no task/no TTY; `1` means the request failed. They need different fixes.

**4. Where does the assistant's reply live in the response object?**

A. In `reply.choices[0].message.content` (and possibly `tool_calls`).

---

# Stage 4: Conversation history

## What this stage is really about

Stage 4 introduces **session state**. The assistant remembers what you just said because *you* keep the messages and send them again. The model has no memory between requests; the harness does.

This is the first appearance of the most important sentence in the course:

> **Messages are the memory.**

The `messages` array is the agent's entire working state. There is no database, no scratchpad file, no summary of what happened earlier. Delete the array and the agent has no idea it ever did anything.

## The four roles

| ROLE | WHO WRITES IT | WHAT IT CARRIES |
|---|---|---|
| `system` | you | The job description: behavioural rules, memory digest (later). |
| `user` | you | The task, or a corrective nudge (the truncation message is also a `user` turn). |
| `assistant` | the model | `content` (its thinking, printed as `think:`) and possibly `tool_calls` — its intentions. |
| `tool` | the harness | One string per call, tagged with the `tool_call_id` it answers. The only way the model learns what happened. |

At Stage 4 there are only the first three. `tool` arrives in Stage 7, but the table is worth learning now because it explains why the array is structured the way it is.

## The session loop

```
const messages = [{ role: 'system', content: SYSTEM_PROMPT }];

while (true) {
  const task = await askTask();
  if (task === null || task === '/quit' || task === '/exit') break;
  if (task === '/new') {
    messages.length = 0;
    messages.push({ role: 'system', content: SYSTEM_PROMPT });
    continue;
  }
  if (!task) continue;

  messages.push({ role: 'user', content: task });
  const reply = await chat({ ...config, messages });
  const answer = reply.choices?.[0]?.message?.content ?? '';
  messages.push({ role: 'assistant', content: answer });
  console.log(ui.answer(answer));
}
```

Three behaviours to notice:

1. **Every turn appends `user` → model call → append `assistant`.** The next request contains the whole conversation, so follow-up questions work.

2. `/new` **clears the conversation but not the process.** The system prompt is re-added; the workspace and (later) memory are untouched.
3. **The loop is still in** `cli.js`**.** That is correct for now. We extract it in Stage 12, after it has grown tools and hooks.

### Why `/new` is not `/reset`

`/new` clears the conversation. It does not delete files, clear memory, or reset configuration. This distinction matters more and more as the course progresses:

- A conversation is *this sitting*.
- The workspace is *the project*.
- Memory is *what survives the process*.

Three different lifetimes, three different commands or files. Confusing them is a common source of "why did it forget?" and "why did it remember that?" bugs.

### One-shot versus interactive

A task passed as an argument stays one-shot: run it, print the answer, exit. A task typed at the prompt starts a session. This is not just UX; it is what the demo and the tests want. A one-shot program is composable in a shell pipeline. A session is for a human exploring.

The session summary at exit (`session over - N turn(s)`) is a small observability feature: it tells you what happened without reading the transcript.

### The context-window seed

Every message you send costs tokens. A long session grows the array until the provider rejects it or your bill becomes surprising. Stage 14 addresses this with `trimHistory`, but the seed is planted here: **history is a resource with a cost.** At Stage 4, just notice that a 50-turn conversation is 50 times the input of a one-shot request.

### Checkpoint

**1. Why does the model "remember" the previous turn if it has no memory?**

A. The harness keeps the `messages` array and sends it again; the model itself has no memory.

**2. What exactly does `/new` clear, and what does it leave alone?**

A. It clears the conversation; the workspace and memory survive.

**3. Why is the loop still in `cli.js` at this stage?**

A. The loop is still simple enough to live in the CLI; extracting it now would hide the mechanics.

**4. What is the difference in lifetime between `messages` and `workspace/`?**

A. `messages` dies with the process; `workspace/` persists on disk.

---

# Stage 5: The system prompt

## What this stage is really about

Stage 5 separates **behavioural rules from control flow**. The loop should not contain the model's job description; that belongs in a prompt builder. The loop should not know whether memory exists; the prompt builder decides what to include.

## Why a module instead of a string literal

A string literal in `cli.js` is fine for one rule. It becomes a problem when:

- the rules depend on which tools exist;
- the prompt contains a memory digest;
- tests need to assert the rules are present;
- a log line must report how many memories were injected.

`prompt.js` centralises "what the model is told" in one place. That is the same design instinct as `ui.js` centralising colour: one decision, one location, no drift.

## The rules a coding agent needs

At Stage 5 the tools do not exist yet, so the prompt is only the behavioural contract. The plan's rules cover:

1. **Look before you write.** Read the file before editing it.
2. **Prefer `create_file` for new files**, `edit_file` for small changes, `write_file` for whole-file replacement.
3. **Copy `old_string` exactly**, including whitespace. The edit tool matches literally.
4. **Run what you wrote.** A file that has not executed is a hypothesis.
5. **Do not invent tool results.** The harness supplies them; you must wait for them.
6. **Be concise.** The user is watching a terminal.
7. **One tool call per step.** Interleaved edits are hard to audit.
8. **Remember useful facts**, not task progress. (Meaningful once Stage 10 exists.)

Why these rules and not a personality? Because each one prevents a concrete failure you will see in the demo:

- Without rule 1, the model edits a file it has not read and guesses the snippet.
- Without rule 3, `edit_file` returns "not found" and the model wastes a step.
- Without rule 4, the model claims success without proof.
- Without rule 5, the model hallucinates output and the loop loses its grounding.
- Without rule 7, a single reply can contain three interleaved edits that are impossible to follow live.

This is prompt engineering as ordinary code: rules in a string, composed by a function, tested like any other behaviour.

`buildSystemPrompt({ memory, log })`

```
export function buildSystemPrompt({ memory = null, log = () => {} } = {}) {
  const rules = systemRules(TOOL_NAMES);
  const digest = memory?.digest?.() ?? '';
  if (!digest) return rules;
  log(ui.dim(`memory ← ${memory.label} - ...`));
  return `${rules}\n\n${digest}`;
}
```

At Stage 5 there is no memory, so the digest is always empty. The signature is deliberately future-proof: the loop can call `buildSystemPrompt({ memory })` from the first day and never change the call site.

The log line matters. When memory is added, the banner will say `memory ← src/memory.md - 5 memories added to the system prompt`. Observability is built in at the point where the decision is made, not reconstructed later from the transcript.

## Tools are passed but empty

`systemRules(tools = TOOL_NAMES)` accepts a tool list so the rules can mention only what exists. At Stage 5, `cli.js` passes `[]` because no tools are built. At Stage 7, the list grows to the read-only tools. At Stage 8, the write tools. At Stage 9, `run_command`.

This is why a generic "you are a coding agent with these tools" prompt is inferior: it names tools the model cannot call, which teaches it to hallucinate calls. **The prompt should describe the program the model is actually attached to.**

## The system prompt is rebuilt, not carried

This is a subtle but crucial habit. In Stage 12, the system prompt will be rebuilt for every task:

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
```

Why rebuild?

- Memory may have been written by the previous task; a carried system message would be stale.
- The tool list may have changed between runs.
- `/new` should restore the prompt without duplicating it.

At Stage 5, simply observe that the system prompt is the *first* message on every turn and that `/new` keeps it. The rebuild pattern becomes explicit in Stage 14.

## Checkpoint

**1. Why keep the model's behavioural rules out of `cli.js`?**

A. Behaviour is a separate decision from control flow; tests and memory need one place to build the prompt.

**2. Why pass an empty tool list at Stage 5 instead of listing all eight tools optimistically?**

A. Naming tools that do not exist teaches the model to hallucinate calls.

**3. What are the two failure modes rule 3 (copy `old_string` exactly) prevents?**

A. It prevents a failed exact match and wasted steps; it also prevents editing the wrong occurrence.

**4. Why is the system prompt rebuilt for each task rather than appended to history?**

A. Rebuilding lets memory saved by the previous task appear in the next task's prompt and avoids stale/duplicate system messages.

---

# Phase B: Give the chatbot hands

**Goal of the phase:** the model can list, read, create, edit, delete and run things — with a fence around writes and a human in the loop for commands.

**What you can do at the end:** ask the agent to create a script, run it, edit it, and verify the edit, while a write outside `workspace/` is refused.

This is the phase where the program stops being a chat client and becomes an agent. It is also the phase where safety stops being theoretical.

---

# Stage 6: Path policy

## What this stage is really about

Stage 6 is the foundation of agent safety, and it begins with a sentence worth putting on a wall:

> **Reads are open; writes are fenced.**

Why asymmetric?

- An agent that cannot read a library, a log, a `package.json` one folder up, or a doc outside its own folder is useless. Reading is how it learns context.
- A write that leaves the workspace is an incident. It can overwrite a configuration, drop a file on the Desktop, or modify something the user did not offer.

So `workspace.resolve()` is strict and used for every write. `workspace.resolveRead()` is permissive and used for every read.

## Why `path.resolve` plus `startsWith` is not enough

The naive containment check is:

```
const abs = path.resolve(root, requested);
if (!abs.startsWith(root)) throw new Error('escape');
```

This fails on Windows in several ordinary-looking ways:

| REQUEST | WHY THE NAIVE CHECK FAILS |
|---|---|
| `C:\Windows\x.txt` | `path.resolve(root, 'C:\\Windows\\x.txt')` returns the absolute path; `startsWith(root)` is false, so it may be caught — but drive-relative forms are subtler. |
| `\\server\share\x` | UNC paths bypass drive-rooted containment entirely. |
| `~/Desktop/x.txt` | `~` is not expanded by `path.resolve`; a later expansion could point anywhere. |
| `%USERPROFILE%\x` | Environment expansion is not traversal, so `..` checks miss it. |
| `Desktop\notes.txt` | If the cwd is inside a user profile, this resolves happily to a real Desktop. |
| `../../secrets.txt` | Caught late by containment, but after other checks have been bypassed. |
| `link-to-home\x` | A junction or symlink passes a string check; the real target may be outside. |

The lesson: **containment is a policy, not a string comparison.** You enumerate the escape shapes you know, expand what the shell would expand, and resolve symlinks before comparing.

## The policy, in order

`workspace.resolve(requested)` rejects:

1. empty or non-string values;
2. absolute paths (`/foo`, `C:\foo`);
3. drive-relative paths (`C:foo`);

4. UNC paths (`\\server\share`, `//server/share`);
5. `~`, `$`, `%` prefixes;
6. a first segment that looks like a home or system folder (`users`, `windows`, `program files`, `desktop`, `documents`, `downloads`, `appdata`, `onedrive`, `library`, `system32`, and similar).

Then it expands variables **before** containment checks:

- `~` → home directory;
- `%VAR%` → Windows environment variable;
- `${VAR}` and `$VAR` → POSIX-style environment variable.

Then it resolves the path against the root and checks containment. The order matters: expanding after the check would let `%USERPROFILE%\x` slip through.

### Symlink hardening: `realResolve`

A junction inside the workspace can point outside it. A string check sees `workspace/link-to-home/x`, which looks contained. The real target is `C:\Users\me\x`.

`realResolve` handles this:

1. Take the absolute path.
2. Walk up to the deepest ancestor that actually exists.
3. Resolve that ancestor with `fs.realpathSync`.
4. Re-append the not-yet-created tail.
5. Re-check containment against the real path.

Why "deepest existing ancestor"? Because the file you are about to create does not exist yet, so you cannot `realpath` it. The closest existing parent is the best available truth.

This is defense in depth: even if a link is created after the check, the next check resolves the link's target. It is not perfect against a race, but it closes the obvious hole.

### Why `PathError`

A dedicated error class lets the sandbox distinguish "the path policy refused this" from "an unexpected I/O error happened". The sandbox converts `PathError` into `ForbiddenError` with an action and a message; everything else is a bug and should surface.

This is a recurring pattern: **classify errors by who can fix them.** A `PathError` is a policy decision; an `EACCES` is an environment problem; a `TypeError` is a programming bug.

### `rel` is the user-facing path

`resolve` returns both `abs` and `rel`:

- `abs` is what the filesystem handler uses;
- `rel` is what the model and the user see.

`rel` is always `/`-separated, even on Windows, and the workspace root is `"."`. Why normalise? Because model output, logs and tests should not vary by platform. A result that says `demo/hello.js` is readable everywhere; `demo\hello.js` is not.

## Checkpoint

**1. Why are reads open but writes fenced?**

A. Reads are how the agent learns context; writes are how it causes incidents.

**2. Why must variable expansion happen before containment checks?**

A. Expansion changes the effective path; checking before expansion lets `%VAR%`/`~` slip through.

**3. What does `realResolve` protect against that a string check does not?**

A. A symlink or junction can point outside the workspace while its name looks contained.

**4. Why is `rel` normalised to forward slashes?**

A. Cross-platform consistency in logs, results and tests.

**5. What error class should a refused path raise, and why not a generic `Error`?**

A. `PathError` separates a policy refusal from an unexpected I/O error so the sandbox can convert it.

---

# Stage 7: Read-only file tools

## What this stage is really about

Stage 7 introduces the first tools the model can invoke, and with them the core protocol of tool calling:

- a **schema** the model reads;
- a **dispatcher** that validates and runs;
- a **string observation** the model reads next.

This is the stage where "everything is messages" becomes concrete. The model emits JSON; the harness returns a string; the string becomes a `tool` message; the loop continues.

## Tool calling is the model's only ability to act

The model never touches the filesystem. It emits something like:

```
{
  "id": "call_1",
  "type": "function",
  "function": {
    "name": "read_file",
    "arguments": "{\"path\":\"demo/hello.js\",\"limit\":50}"
  }
}
```

The harness:

1. reads `name`;
2. parses `arguments` (a JSON *string*, not an object);
3. validates types and required fields;
4. dispatches to a handler;
5. returns a string;
6. appends `{ role: 'tool', tool_call_id, content }`.

The model sees the result on the next request. That is the whole mechanism.

## The schema is the prompt

When the model chooses a tool it reads nothing but these descriptions. That makes them the highest-leverage text in the codebase — and the first place to fix a model that keeps picking the wrong tool.

```
{
  type: 'function',
  function: {
    name: 'create_file',
    description: 'Create a NEW file with the given content. Fails if the file already exists - use edit_file or
write_file for existing files.',
    parameters: {
      type: 'object',
      properties: {
        path: { type: 'string', description: 'File path relative to the workspace root' },
        content: { type: 'string', description: 'Full file content' },
      },
      required: ['path', 'content'],
      additionalProperties: false,
    },
  },
}
```

Note what the description does:

- states the **precondition** ("fails if the file already exists");
- names the **alternative** ("use edit_file or write_file");
- keeps the parameter descriptions short and concrete.

`additionalProperties: false` discourages invented arguments. It is not a security boundary (the dispatcher validates anyway), but it is a prompt signal.

## Validation before dispatch

```
function validate(name, rawArgs) {
  const schema = SCHEMAS_BY_NAME[name];
  let args;
  try {
    args = typeof rawArgs === 'string' ? JSON.parse(rawArgs) : rawArgs;
  } catch {
    return { error: `ERROR: ${name} arguments are not valid JSON` };
  }
  // type checks, integer checks, required fields
  return { args };
}
```

Why validate?

- Models send `"limit": "20"` instead of `20`. A type error two frames down is confusing; an immediate `ERROR: "limit" must be an integer, got string` is actionable.
- Models omit required fields. Say which one.
- Models invent arguments. Reject unknown keys when `additionalProperties` is false.

The error string is itself a prompt. It should say what went wrong and what to do, not just "invalid".

## A handler returns a string, always

This is the fourth principle: **errors are observations, not crashes.**

Three rules hold for every tool:

1. **Return a string.** Success, denial, unexpected exception — everything is text the model reads on the next step.
2. **Parse and type-check arguments before dispatch.** Do not let a bad type become a crash.
3. **Unknown names are survivable.** A hallucinated tool name gets a helpful list and the loop continues.

A thrown error would escape the loop and end the run. A returned `ERROR: …` is an observation the model can act on. The distinction is not cosmetic; it is the difference between a one-step repair and a dead session.

### `list_files`: bounded and sorted

A recursive listing can explode. The handler bounds it:

- `path` defaults to `'.'`;
- `depth` defaults to `3`, max `6`;
- skip `.git`, `node_modules`, `.venv`, `__pycache__`;
- directories print `<rel>/`, files print `<rel> (<size>b)`;
- stop at `depth` and `200` lines;
- end with `<dirs> folder(s), <files> file(s) under <start.rel>`.

Why this shape?

- **Sorted** output is deterministic, which makes tests and human reading easier.
- **Skipping dependency folders** keeps the answer about the user's code, not `node_modules`.
- **A summary line** tells the model it saw a complete or truncated view.
- **A line cap** prevents a giant listing from consuming the context window.

Missing folder → `ERROR: no such folder: <rel>`. That is a valid observation, not a crash.

### `read_file`: numbered, paged, honest about truncation

Parameters:

- `path` (required);
- `offset` (default `1`);
- `limit` (default `200`, max `2000`).

Behaviour:

- missing file → `ERROR: no such file: <rel>`;
- directory → `ERROR: <rel> is a folder - use list_files`;
- header: `"<rel> (<total lines>)"`;
- numbered lines right-aligned to width 4 followed by `|`;
- append `"\n... [<n> more lines]"` when truncated;
- clamp `offset` to at least `1`, `limit` to `1..2000`.

Why line numbers? Because the model will quote fragments back to `edit_file`, and line numbers make it possible to talk about location. Why a total line count? Because "you are seeing 50 of 800 lines" changes the model's strategy. Why a truncation marker? Because silently showing the first 200 lines teaches the model that the file is 200 lines long.

## The manual tool loop in `cli.js`

At Stage 7 the loop is still in the CLI, and it is deliberately manual:

```
let reply = await chat({ ...config, messages, tools: TOOL_SCHEMAS });

while (reply.choices[0].message.tool_calls?.length) {
  const message = reply.choices[0].message;
  messages.push(sanitizeAssistant(message));

  for (const call of message.tool_calls) {
    console.log(ui.tool(call.function.name, call.function.arguments));
    const result = await runTool(call.function.name, call.function.arguments, ctx);
    console.log(ui.result(result));
    messages.push({
      role: 'tool',
      tool_call_id: call.id,
      content: result,
    });
  }

  reply = await chat({ ...config, messages, tools: TOOL_SCHEMAS });
}
```

This is the ReAct pattern in about twenty lines. It teaches the shape before Stage 12 formalises it. The important details:

- `tool_calls` **is an array.** The plan asks for one call per step in the prompt, but the protocol allows several; dispatch them in order.
- **Every** `tool_call_id` **must get a matching** `tool` **message.** A provider rejects a conversation with an unanswered call.
- **The result is a string, even when it is an error.** Append it anyway.
- **Append the assistant message before the tool results.** The order is: assistant intention, then observations.

## Why reads outside the workspace are allowed

`read_file` uses `resolveRead`, so `../package.json` works. That is a feature, not an oversight: the agent often needs to understand the package it lives in. The fence is about writes.

The asymmetry has a teaching benefit: in the demo, the model reads `../package.json` immediately after a write escape is refused. The audience sees the policy boundary, not a blanket "no".

## Checkpoint

**1. Why must a tool handler return a string rather than throw?**

A. A throw ends the run; a string lets the model recover on the next step.

**2. Why is `arguments` a JSON string, and what happens if you pass an object?**

A. The protocol defines `arguments` as a JSON string; passing an object means the dispatcher must normalise it.

**3. Why does `read_file` include a total line count and a truncation marker?**

A. They tell the model it saw a truncated view and that the file is larger.

**4. What happens if the model calls an unknown tool?**

A. It gets a helpful list of available tools and the loop continues.

**5. Why is the tool loop still in `cli.js` at this stage?**

A. It is still teaching the ReAct shape; formal extraction comes in Stage 12.

---

# Stage 8: Sandbox and write tools

## What this stage is really about

Stage 8 adds **defense in depth** around writes, and it establishes the one place in the codebase that prints `FORBIDDEN`. It also teaches the honest limits of a sandbox built out of path checks.

There are two layers here, and they answer different questions:

- `workspace.js`: *does this path stay inside the root?*
- `sandbox.js`: *may the agent change this file at all?*

The sandbox delegates path policy to the workspace, converts refusals into a loud, typed error, and centralises the count of refusals.

## Why a separate sandbox layer?

You could call `workspace.resolve()` directly in every write handler. Why add a wrapper?

1. **One place to count refusals.** `sandbox.refusals` is a useful statistic and a test hook.
2. **One place to print `FORBIDDEN`.** If three handlers each format their own refusal, the output drifts and the tests get brittle.
3. **One place to encode action semantics.** `save`, `edit`, `delete` are different actions with different rules (for example, deleting the root is always refused).
4. **One place to convert `PathError` into `ForbiddenError`.** The tool layer can then return `FORBIDDEN: ...` consistently.

### `ForbiddenError` carries context

```
export class ForbiddenError extends Error {
  constructor(action, requested, reason) {
    super(`${action.toUpperCase()} ${requested} - ${reason}`);
    this.name = 'ForbiddenError';
    this.code = 'FORBIDDEN';
    this.action = action;
    this.requested = requested;
    this.reason = reason;
  }
}
```

Why fields as well as a message?

- The tool layer can return `FORBIDDEN: ${err.message}` without re-parsing a string.
- Tests can assert `err.code === 'FORBIDDEN'` rather than matching prose.
- The UI can print a structured line.

### `check`: the single gate

```
check(action, requested) {
  if (!ALLOWED.has(action)) throw new TypeError(`unknown sandbox action "${action}"`);

  let target;
  try {
    target = this.workspace.resolve(requested);
  } catch (err) {
    if (err instanceof PathError) this.forbid(action, requested, err.message);
    throw err;
  }

  if (action === ACTIONS.DELETE && target.rel === '.') {
    this.forbid(action, requested, 'the sandbox root itself cannot be deleted');
  }
  return target;
}
```

Notice the two different failure modes:

- An **unknown action** is a programming bug → `TypeError`. It should not be caught as a refusal.
- A **path policy violation** is an expected refusal → `ForbiddenError`, printed and returned to the model.

This distinction recurs: expected failures are observations; unexpected failures are bugs.

### `save`, `remove`, and the root guard

- `save(requested, content)`: `check(SAVE)`, create parents, write, return `{ existed, bytes }`.
- `remove(requested)`: `check(DELETE)`, `rmdirSync` for empty directories, `unlinkSync` for files; missing returns `{ missing: true }`.
- Deleting the sandbox root itself is refused.
- `summary()` returns `"save, edit, delete - only inside <root>"`.

Why refuse the root? Because `delete_file(".")` would remove the entire workspace, which is never what a user wants from a single tool call. It is a cheap guard against a catastrophic mistake.

### Write tools: three different jobs

The three file-writing tools are not redundant. Each encodes a different intent and a different safety property:

| TOOL | INTENT | SAFETY PROPERTY |
|---|---|---|
| `create_file` | Make a new file | **Fails if it already exists** — forces the model to edit rather than clobber. |
| `write_file` | Replace a whole file | Explicitly allowed to overwrite; creates parents. |
| `edit_file` | Change a fragment | Exact match; refuses ambiguity. |

Why does `create_file` refuse to overwrite? Because "create" should not silently destroy. A model that wants to replace a file should say so with `write_file`, and the transcript then shows the intent. A model that re-runs a script and calls `create_file` gets an error it can recover from, rather than wiping a user's work.

### `edit_file`: the interesting one

Seven of the eight tools are obvious. `edit_file` is where an agent's output is most likely to be subtly wrong, and where good harness design turns that into a recoverable moment.

```
edit_file(ws, args) {
  const target = ws.resolve(args.path);
  if (!fs.existsSync(target.abs)) return `ERROR: no such file: ${target.rel}`;

  const oldStr = String(args.old_string);
  const newStr = String(args.new_string);
  if (oldStr === '') return 'ERROR: old_string must not be empty';

  const source = fs.readFileSync(target.abs, 'utf8');
  const count = source.split(oldStr).length - 1;

  if (count === 0) {
    return `ERROR: old_string not found in ${target.rel}. Read the file and copy the snippet exactly, including
whitespace.`;
  }
  if (count > 1 && !args.replace_all) {
    return `ERROR: old_string appears ${count} times in ${target.rel}. Add surrounding context to make it unique, or
pass replace_all: true.`;
  }

  const updated = args.replace_all
    ? source.split(oldStr).join(newStr)
    : source.replace(oldStr, () => newStr); // function form: no $1 surprises

  fs.writeFileSync(target.abs, updated, 'utf8');
  return `edited ${target.rel}: ${args.replace_all ? count : 1} replacement(s), first at line ${lineNo}`;
}
```

## Every failure mode, and why it is a message

| SITUATION | WHAT COMES BACK | WHY THAT IS RIGHT |
|---|---|---|
| Snippet is not in the file | `ERROR: old_string not found … copy the snippet exactly, including whitespace.` | The model hallucinated whitespace. The message says what to do next, so the following step usually fixes it. |
| Snippet appears 4 times | `ERROR: old_string appears 4 times … pass replace_all: true` | Silent first-match replacement would edit the wrong function. Ambiguity must be an error, not a guess. |
| Snippet is `""` | `ERROR: old_string must not be empty` | An empty search string would otherwise "match" everywhere. |
| Replacement contains `$1` | replaced literally | `String.replace` interprets `$1` when the second argument is a string; the callback form removes the footgun. |
| File does not exist | `ERROR: no such file: …` | Points at the missing step without being asked. |

The `$1` detail is a classic JavaScript trap worth understanding:

```
'hello world'.replace('world', '$&!');   // 'hello world!'
'hello world'.replace('world', () => '$&!'); // 'hello $&!'
```

When the replacement is a string, `$&`, `$1`, `$'` and friends are special. When it is a function, the return value is used literally. For a code-editing tool, literal is the only safe choice: a replacement containing `$1` is almost certainly code, not a backreference.

### The `FORBIDDEN` contract

A refused write:

1. prints exactly one bold red line: `⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt`;
2. throws `ForbiddenError`;
3. is converted by the tool dispatcher into `FORBIDDEN: ...` text;
4. is returned to the model as an observation;
5. is appended to the message array like any other result.

The model can then choose another path. The loop does not crash. The user sees one loud line. That is the full contract.

### Honest limitations

This sandbox is a **path policy**, not an operating-system sandbox. It does not isolate processes, restrict syscalls, or prevent a command from writing outside the workspace. Stage 9 adds a command-line scan, but that is still a seatbelt, not a container. Say this out loud in your documentation and your demo. Implying the demo is safe by construction is worse than admitting its limits.

### Checkpoint

**1. What are the two questions `workspace.js` and `sandbox.js` answer?**

A. `workspace.js`: is this path contained? `sandbox.js`: may the agent change this file at all?

**2. Why is an unknown sandbox action a `TypeError` rather than a `ForbiddenError`?**

A. Unknown action is a programming bug; a path refusal is an expected policy outcome.

**3. Why does `create_file` refuse to overwrite, while `write_file` does not?**

A. "Create" should not silently destroy; replacement should be explicit via `write_file`.

**4. Why does `edit_file` use the callback form of `String.replace`?**

A. The callback form treats `$1`, `$&` literally, which is what code replacement needs.

**5. Why is deleting the sandbox root refused separately from the path check?**

A. Deleting the root is a catastrophic case that deserves its own explicit guard and message.

---

# Stage 9: Command execution

## What this stage is really about

Stage 9 gives the model a shell, which is both the most useful and the most dangerous tool in the harness. The theory here is about **layered policy**:

1. a hard deny-list that no policy can override;
2. a command-line scan for visible write targets outside the workspace;
3. a configurable policy (`deny` / `ask` / `allow`);
4. a human approval prompt;
5. a restricted child environment;
6. output clipping and timeouts.

No single layer is sufficient. Together they make the common cases safe and the remaining risk explicit.

## Why a shell at all?

Because "run what you wrote" is the difference between a plausible edit and a verified one. Exit codes are how the loop closes: a file that has not executed is a hypothesis. The model needs to run tests, linters, and scripts, and a shell is the universal interface to them.

But a shell is also a general-purpose escape hatch. It can copy, delete, download, and execute arbitrary code. That is why it gets the most policy.

## The hard deny-list

Eight patterns are checked **before** the policy is consulted, so `allow` cannot unblock them:

| PATTERN | REFUSED BECAUSE |
|---|---|
| `rm -rf /`, `rm -rf ~` | recursive delete of a root path |
| `del /s`, `del /q` | recursive or quiet Windows delete |
| `format`, `diskpart` | disk formatting |
| `shutdown`, `reboot` | machine power control |
| `git push --force`, `git push -f` | force push |
| `curl … \| sh`, `wget … \| bash` | piping the internet into a shell |
| `:(){ :\|:& };:` | fork bomb |
| `> /dev/sda` | raw disk write |

These are not a security boundary. They are a seatbelt against a model that has read too many bad shell examples. The point is to make the catastrophic cases impossible, not to make every case safe.

Why check before policy? Because a user who types `--yes` is approving ordinary commands, not `format C:`. The hard deny-list is a statement that some commands are never part of the task.

## Scanning the command line for writes

The file tools were never the whole story. `copy hello.js %USERPROFILE%\Desktop\hello.js` leaves the sandbox without touching a file tool. The command line says where it is writing, so `sandbox.checkCommand()` reads it first.

The scanner tokenises the command, then extracts write targets by verb family:

| COMMAND FAMILY | WRITE TARGET |
|---|---|
| `copy`, `cp`, `move`, `mv`, `xcopy`, `ren`, `rename`, `ln`, `install`, `scp`, `rsync` | last positional (require at least two) |
| `robocopy` | positional argument 2 |
| `del`, `erase`, `rd`, `rmdir`, `rm`, `md`, `mkdir`, `touch`, `tee`, `unlink`, `truncate`, `shred` | every positional |
| `curl` | after `-o`, `--output`, `-O`, `--remote-name` |
| `wget` | after `-O`, `--output-document` |
| PowerShell `out-file`, `set-content`, `add-content`, `new-item`, `ni`, `remove-item`, `copy-item`, `move-item`, `rename-item` | after `-destination`, `-dest`, `-path`, `-filepath`, `-literalpath` |
| `>` / `>>` redirection | the following token (`2> file` handles a stream id) |

Then each target is passed through `workspace.resolve()`, the same policy a `write_file` uses. A refusal prints the same `FORBIDDEN` line and throws before the command runs.

## Tokenisation is not parsing

`tokenizeCommand(text)` splits on whitespace while honouring single and double quotes, and emits shell operators as separate tokens: `>`, `>>`, `<`, `|`, `||`, `&`, `&&`, `;`. Quotes are stripped.

This is deliberately not a full shell parser. It handles the common cases and fails closed on ambiguity. Two details matter:

- **Quoted arguments with spaces stay one token.** Otherwise a filename with a space looks like two positional arguments.
- **A switch is `--flag`, `-f`, or Windows `/x`.** A POSIX path such as `/home` is not a switch. Distinguishing them prevents a path from being skipped as an option.

## Policy layers

| `CMD_POLICY` | BEHAVIOUR | WHEN YOU WOULD USE IT |
|---|---|---|
| `ask` (default) | Every command stops and waits for `y/N`. `n` returns `DENIED: the user did not approve this command.` | Live demos and any run you are watching. The pause is the point. |
| `allow` | Commands run without asking. Set by `--yes`, `--allow-commands`, or `CMD_POLICY=allow`. | The scripted demo, CI, or a task you already trust. |
| `deny` | No command runs at all; the model gets a denial and can still edit files. | Read-mostly tasks, or a run with no terminal. |

Order of checks in `run_command`:

```
trim → empty? → hard deny-list → sandbox.checkCommand → policy
  → deny: return DENIED
  → allow: proceed
  → ask: confirm via readline
  → execute
```

Notice that the sandbox scan happens **before** approval. A `copy` to the Desktop never reaches the prompt. This is better UX and better security: do not ask a human to approve something you already know is forbidden.

### The non-TTY rule

With no terminal attached (piped input, CI, a scheduled run), the `ask` policy degrades to **deny**. An unanswerable prompt must never become an approval.

This is why `confirm` defaults to `async () => 'deny'`. The default is the safe answer, and a caller must opt into asking. It is a small design decision with a large consequence: the harness is safe by default in automation.

### The child environment allow-list

The child process must not inherit the harness environment. If it did, a command could print `OPENROUTER_API_KEY`, and the model would see it in the observation. The demo checks exactly this.

`safeEnv(source)` keeps only:

`PATH`, `PATHEXT`, `HOME`, `USERPROFILE`, `TMPDIR`, `TEMP`, `TMP`, `SYSTEMROOT`, `COMSPEC`, `SHELL`, `LANG`, `LC_ALL`, `TERM`.

Why these?

- `PATH` and `PATHEXT` are needed to find commands.
- `HOME` / `USERPROFILE` are needed by many tools.
- Temp variables are needed by compilers and installers.
- `SYSTEMROOT` / `COMSPEC` are needed by `cmd.exe`.
- `SHELL`, `LANG`, `LC_ALL`, `TERM` are needed for predictable Unix behaviour.

Everything else is dropped, including API keys, tokens, and unrelated configuration. This is least privilege at the process boundary.

### Execution details

```
exec(command, {
  cwd: workspace.root,
  timeout: 30_000,
  maxBuffer: 1024 * 1024,
  windowsHide: true,
  encoding: 'utf8',
  env: safeEnv(process.env),
});
```

- `cwd: workspace.root` makes relative paths predictable.

- `timeout: 30_000` stops a hanging command from hanging the run.
- `maxBuffer: 1 MB` stops a chatty command from exhausting memory.
- `windowsHide: true` avoids a console window flash on Windows.
- `encoding: 'utf8'` makes the result a string.

### Result format

```
exit code <n>
stdout:
<first lines>
stderr:
<first lines>
```

Rules:

- include `stdout:` only when non-empty;
- include `stderr:` only when non-empty;
- add `(no output)` when both are empty;
- clip each stream to 6,000 characters, appending `... [<n> chars truncated]`;
- on timeout, append `[tool] command timed out after 30s` to stderr and return a non-zero code.

Why include the exit code first? Because it is the verdict. The model should be able to decide "did this work?" before reading the output. Why clip? Because a test suite can print megabytes, and the model does not need all of it to act.

### Why `run_command` is the most important teaching tool

It demonstrates four principles at once:

- **Exit codes close the loop.** `exit code 0` versus `1` is how the model knows whether its code works.
- **Human in the loop.** The default is to ask.
- **Least privilege.** The environment is allow-listed.
- **Bounded autonomy.** There is a timeout and an output cap.

It is also the tool that makes the sandbox's limitations honest. The command-line scan is not isolation. A one-liner can still write anywhere. Say so.

### Checkpoint

**1. Why is the hard deny-list checked before the policy?**

A. Some commands are never part of a legitimate task; policy cannot override that.

**2. Why is the command-line write scan done before the human approval prompt?**

A. To avoid asking a human to approve something already known to be forbidden.

**3. Why does `ask` degrade to `deny` when stdin is not a terminal?**

A. An unanswerable prompt must never become an approval.

**4. What does the child environment allow-list protect against?**

A. It prevents API keys and unrelated secrets from leaking into tool output.

**5. Why is the exit code the first line of the result?**

A. It is the verdict: did the command succeed? The output is evidence.

**6. What can still escape the sandbox even with all these layers?**

A. A script or one-liner can still write anywhere; the scan reads the command line, not the code.

---

# Phase C: Memory and observability

**Goal of the phase:** the agent can remember facts across runs, and you can see exactly what it is doing at every lifecycle moment.

**What you can do at the end:** save a durable note, find it again in a later process, and print a trace of all nine lifecycle hooks.

---

# Stage 10: Long-term memory

## What this stage is really about

Everything so far lives and dies inside one process. `messages` is working memory; when the process exits it is gone — including lessons that were expensive to learn ("the tests are offline", "this repo spells it Brighton", "force-push is refused").

Stage 10 adds the smallest thing that actually persists: **a markdown file** and two tools that read and write it.

> **Memory is a file, not a mystery.**

No embeddings, no vector store, no summarising, no write on every turn. One file a human can read, diff and delete a line from. "Why did it remember that?" is a `cat`, not an investigation.

## Why a file, not a database?

- **No cache, no drift.** Every method reads the file. A hand edit, a deletion or a `git checkout` is picked up on the next call. A memory module with a cache is a memory module that lies.
- **It diffs like code.** `src/memory.md` appears in `git status`, diffs cleanly, and can be reviewed in a pull request.
- **You can audit it.** `node src/cli.js --memory` prints the file and the exact block the prompt received.
- **Nothing to migrate.** One bullet per memory; the bullet *is* the record.

This is not the final word in memory architecture. It is the first thing that works, and every real system grows past it. Starting here teaches the boundary: what persists, what costs context, and what a human can trust.

## The format

```
# memory

Long-term notes this agent keeps between runs. One bullet per memory:
`- [tag] text _(YYYY-MM-DD)_` where tag is fact | pref | fix | gotcha.

This file IS the store - edit it by hand, delete a wrong line, or commit it.

- [fact] The sandbox demo script lives at demo/hello.js and is safe to regenerate with create_file. _(2026-10-28)_
- [pref] The user wants console output, not files, for anything under three lines. _(2026-10-28)_
```

Tags:

| TAG | MEANING |
|---|---|
| `fact` | A stable fact about the project or environment. |
| `pref` | A user preference. |
| `fix` | A known repair or workaround. |
| `gotcha` | A trap that costs a step if forgotten. |

Why tags? They give `recall` a second dimension to search and they encourage the model to classify what it is saving. They also make the file scannable for a human.

### Parsing is forgiving on purpose

```
export function parseMemory(source) {
  const entries = [];
  for (const line of String(source ?? '').split(/\r?\n/)) {
    const m = line.match(
      /^\s*[-*]\s*\[([a-z]+)\]\s*(.+?)\s*(?:_\((\d{4}-\d{2}-\d{2})\)_)?\s*$/i,
    );
    if (!m) continue;
    const text = normalise(m[2]);
    if (!text) continue;
    entries.push({ tag: m[1].toLowerCase(), text, date: m[3] ?? 'undated' });
  }
  return entries;
}
```

Design choices:

- accept `-` or `*` bullets;
- accept an optional date (hand-written notes may not have one);
- normalise whitespace in the text;
- ignore prose, headings, and comments.

A parser that rejects the whole file on one malformed line would be hostile to the human editor. A parser that skips what it does not understand is a parser you can live with.

### The `Memory` class

```
new Memory({
  root,
  file,
  maxEntries = 60,
  maxEntryChars = 300,
  rememberChars = 2000,
  promptChars = 1200,
  promptEntries = 12,
  now = () => new Date(),
});
```

Every number is a bound, and every bound has a reason. The next sections explain them.

## Writing: `remember`

```
remember(rawText, { tag = 'fact' } = {}) {
  const text = normalise(rawText).slice(0, this.options.rememberChars);
  const entry = {
    tag,
    text: text.slice(0, this.options.maxEntryChars),
    date: today(this.now()),
  };

  const entries = this.entries();
  const existing = entries.findIndex(
    (e) => e.text.toLowerCase() === text.toLowerCase(),
  );
  if (existing !== -1) {
    entries[existing] = { ...entries[existing], date: entry.date, tag: entry.tag };
    this.save(entries);
    return { ok: true, duplicate: true, entry: entries[existing], total: entries.length };
  }

  entries.push(entry);
  const dropped = Math.max(0, entries.length - this.options.maxEntries);
  if (dropped) entries.splice(0, dropped);
  this.save(entries);
  return { ok: true, entry, dropped, total: entries.length };
}
```

Three behaviours worth teaching:

1. **Duplicates refresh the date instead of appending.** Saying the same thing again is not new information.
2. **The oldest entries are dropped at `maxEntries`.** Like `maxSteps`, the file is bounded on purpose.
3. **The text is truncated to `maxEntryChars`.** No single note can dominate the digest.

## Reading: eager and lazy

The two access paths cost different things, so the harness has both.

| PATH | WHAT IT IS | WHAT IT COSTS |
|---|---|---|
| **Eager** — `digest()` | The newest 12 entries, capped at 1200 characters, appended to the system prompt. | A fixed number of characters. It cannot grow with the file, so it cannot quietly eat the context window. |
| **Lazy** — `recall` | A substring search over the whole file, when the model asks for it. | One tool call, and only when the task needs it. |

The digest always says what it left out:

```
## What you remember from earlier runs
These notes live in memory.md and were learned before this task started.
Treat them as strong hints, not facts to trust blindly: the code may have moved on since.
- [gotcha] create_file refuses to overwrite, so re-running the demo fails at step 2. _(2026-10-24)_
- [fix] The demo script lives at demo/hello.js; delete it before a clean demo run. _(2026-10-28)_
- [pref] The user wants console output, not files, for anything under three lines. _(2026-10-28)_
(9 older memories not shown - search memory.md with the recall tool)
```

That last line is the difference between "this is all I know" and "this is what fits — ask for the rest". It is one line of code and a large improvement in honesty.

## Integration: four lines in the loop

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  { role: 'user', content: task },
];
const ctx = { workspace, commandPolicy, memory, confirm };
```

No new message role, no special channel, no vector store. Memory is text in the one message the model always receives.

The actual decision lives in `prompt.js`:

```
const digest = memory.digest();
if (!digest) return rules;
return `${rules}\n\n${digest}`;
```

"What the model is told" is one decision made in one place: the rules, then the digest, then a log line saying how many memories were carried.

## The two tools

`remember`

- params: `text` (required), `tag` (optional);
- no memory store → `ERROR: no memory store in this run (started with --no-memory)`;
- empty → `ERROR: nothing to remember - the note was empty`;
- duplicate → `already remembered in <label> (<total> total), date refreshed: <formatted entry>`;
- new → `remembered in <label> (<total> total[, dropped the N oldest]): <formatted entry>`.

`recall`

- params: `query` (optional), `limit` (default 20);
- no memory → same error;
- no hits → `no memories in <label> match "<query>" (<size> stored)`;
- hits → formatted entries.

Both are ordinary tools: they appear in the transcript, they return strings, and they are subject to the same validation and error handling as file tools.

## Why `remember` is not automatic

Nothing writes memory at the end of a run. A model that saved a memory every step would be filling your context with its own diary. The system prompt says: durable facts only, never task progress. The write is a deliberate tool call, visible in the transcript and auditable.

## The caps

| KNOB | DEFAULT | WHAT IT BOUNDS |
|---|---|---|
| `MEMORY_MAX_ENTRIES` | 60 | the file — the oldest entries are dropped when it is reached |

| KNOB | DEFAULT | WHAT IT BOUNDS |
|---|---|---|
| `MEMORY_PROMPT_CHARS` | 1200 | the injected block; `0` turns injection off and leaves `recall` working |
| `maxEntryChars` | 300 | one memory, so no single note can dominate |
| `rememberChars` | 2000 | one `remember` call |
| `promptEntries` | 12 | how many of the newest appear in the digest |

The most interesting combination is `MEMORY_PROMPT_CHARS=0`: the harness changes from "stuff what fits" to "retrieve when needed", with no code change. That is a design parameter, not a feature flag bolted on later.

## CLI side-doors

- `--memory`: prints the store, the digest, and the raw file, then exits `0`.
- `--no-memory`: disables memory for that run.
- `MEMORY_FILE`: points at a different store (an absolute path shares one store across checkouts).

The side-door is important for trust. "What does it remember about me?" should be a command, not an interrogation of the model.

## Checkpoint

**1. Why does every `Memory` method read the file instead of using a cache?**

A. A cache can disagree with the file; reading every time makes hand edits authoritative.

**2. Why does a duplicate refresh the date instead of appending?**

A. Repeating a note is not new information; refreshing the date keeps recency without duplicating.

**3. What is the difference between the eager digest and lazy `recall`, and why have both?**

A. Eager is bounded and always present; lazy is unbounded search on demand. Both are needed to control cost while keeping access.

**4. Why does the digest state how many memories were hidden?**

A. Honesty: the model should know it is seeing a bounded slice and can ask for the rest.

**5. What does `MEMORY_PROMPT_CHARS=0` change, and what does it leave working?**

A. It disables injection but leaves `recall` working — retrieve-on-demand instead of stuff-what-fits.

**6. Why is a memory write a tool call rather than an automatic end-of-run action?**

A. Automatic writes would fill context with a diary; a visible tool call is auditable and intentional.

# Stage 11: Lifecycle hooks

## What this stage is really about

The loop is fixed on purpose — that is what makes it readable — but a fixed loop is not *extensible*. "Log every command", "never let it touch `.env`", "tell me when it gives up": every one of those used to mean editing `agent.js`, and every one of those edits is a chance to break the loop for everybody.

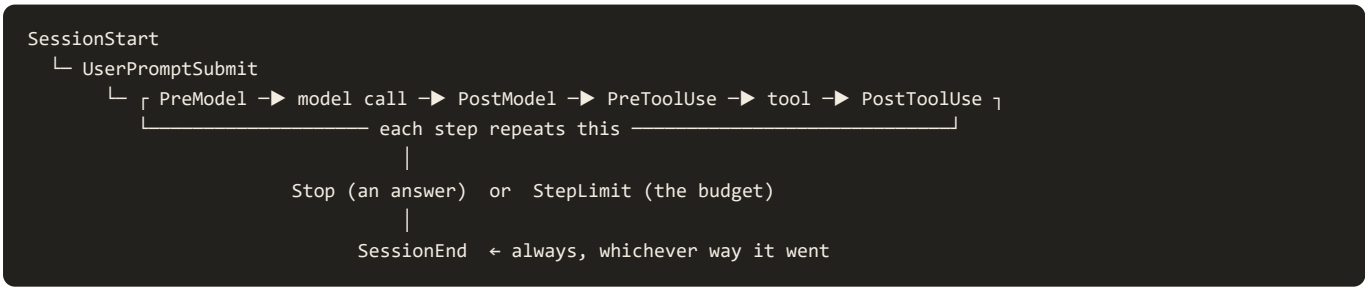
A **hook** is a function attached to a named moment in the loop. The loop calls all nine moments, and your code never edits the loop at all.

This is the chapter that answers the question an audience always asks: *"So how would you make this real?"* Permissions, audit logs, redaction, compaction, notifications — they are all the same mechanism: a named pause plus a decision.

## The nine moments

| # | HOOK | ICON | COLOUR | FIRES | WHAT A HOOK MAY DO |
|---|---|---|---|---|---|
| 1 | `SessionStart` | 🚀 | cyan | once, before the first model call | **refuse to start** — or post a note |
| 2 | `UserPromptSubmit` | ✍️ | blue | once, as the task enters the loop | **rewrite the task**, or block the run |
| 3 | `PreModel` | 🧠 | magenta | before every model call | observe: the request is assembled, not sent |
| 4 | `PostModel` | 💬 | cyan | after every reply, before dispatch | observe: answer or tool call, visible first |
| 5 | `PreToolUse` | 🛠️ | bright yellow | before every tool call | **block** — the tool never runs |
| 6 | `PostToolUse` | 📦 | green | after every tool call | **replace** the observation the model reads |
| 7 | `Stop` | 🏁 | bright green | the model answered and the loop is ending | observe: summary, cost, notification |
| 8 | `StepLimit` | ⏳ | yellow | the loop hit `maxSteps` | observe: escalate, dump the transcript |
| 9 | `SessionEnd` | 🔒 | red | always — finished or not | observe: the last code to run in the process |

## One run, in order




The hooks sit exactly where the loop pauses. Nothing is interleaved by magic. If you add a tenth hook, add it where the code already pauses; a hook for something the loop does not do is documentation, not an extension point.

## Why these nine?

Because the nine are the places the loop *already* pauses:

- before it starts,
- before the first request,
- before and after each reply,
- before and after each tool call,
- on success,
- on exhaustion,
- and at the end.

A hook system should mirror the control flow you have, not invent a control flow you wish you had.

## The contract

A hook is called with a payload object and its **return value** decides what happens:

```
return undefined;                      // observe only - the run is untouched
return { note: 'checked' };            // say something; still untouched
return { task: '…and run the tests' }; // UserPromptSubmit: rewrite the task
return { replace: 'sk-or-[redacted]' }; // PostToolUse: change what the model reads
return { block: true, reason: 'no' };   // SessionStart/UserPromptSubmit/PreToolUse: refuse
```

That is the whole API, deliberately three verbs wide: **observe**, **rewrite**, **block**.

## Four hooks can change the run

- `SessionStart`: refuse to start.
- `UserPromptSubmit`: rewrite the task or block the run.
- `PreToolUse`: block a tool call.
- `PostToolUse`: replace the observation.

The other five only watch. That asymmetry is what makes hooks a safe first extension point: most of them cannot break anything.

## Blocking is an observation, not an exception

When `PreToolUse` blocks a call, the tool is never dispatched *and* the model is told why:

```
DENIED: a PreToolUse hook blocked this call (no pushes from the sandbox). Try another approach.
```

The loop cannot crash on a "no", and the model gets its next turn to choose differently. This is principle 4 applied to permissions: **a refusal is an observation.**

## A throwing hook is reported and ignored

One broken hook must not take the loop with it. The handler is caught, printed as `hook PostToolUse (…) threw: … - ignored`, and the next handler runs.

Why is this essential? Because every `tool_call_id` must still get a matching tool message, or the next request is invalid. A crashing hook that skipped message appending would corrupt the conversation.

This is a general design rule for extension points: **isolate the extension.** The core loop's invariants are more important than any individual plugin.

### The registry

```
export class Hooks {
  on(name, fn, { id, once } = {}) { /* validate name, store handler */ }
  off(name, id) { /* detach */ }
  count(name) { /* how many */ }
  has(name) { /* any attached? */ }
  attached() { /* all events with handlers */ }
  async run(name, payload = {}) { /* call in order, collect decisions */ }
}
```

`run` returns one object:

```
{
  event, ran, blocked, reason, task, replace, notes, errors
}
```

The loop reads `blocked`, `task`, `replace`, and otherwise ignores the rest.

Why collect into one object rather than a boolean? Because a single hook system must express four different decisions (note, rewrite, replace, block) without the loop learning four APIs.

### First block wins

```
if (decision.block) {
  out.blocked = true;
  out.reason = decision.reason ? String(decision.reason) : `${name} blocked this step`;
  break;
}
```

Why stop at the first block? Because continuing would let a later hook "unblock" a refusal, which is confusing and unsafe. A block is final for that moment.

### `traceHooks`

`traceHooks({ log, detail, explain })` returns a `Hooks` instance with one observer per event. It prints:

```
  🛠  [HOOK:PreToolUse] before every tool call - run_command {"command":"node demo/hello.js"}
```

The trace is on by default: every run prints each lifecycle moment as one line, tagged `[HOOK:Name]`, followed by what that moment is *for* and what this firing carried. `--no-hooks` silences it; `--hooks` prints the catalogue.

The catalogue is generated from the same `HOOK_EVENTS` array the loop fires, so the documentation and the console cannot drift apart. That is a small but powerful idea: **generate the teaching material from the code.**

### A permission layer and a redactor, in twelve lines

This is a real, runnable use of the API:

```
import { runTask } from './agent.js';
import { Hooks, printHookCatalogue } from './hooks.js';

const hooks = new Hooks({ log: console.log });

// 1. A permission layer. The model asks; the hook decides.
hooks.on('PreToolUse', ({ name, args }) => (
  name === 'run_command' && /\bgit push\b/.test(args)
    ? { block: true, reason: 'no pushes from the sandbox' }
    : undefined
));

// 2. Redaction. Whatever a command printed, a key never reaches the model.
hooks.on('PostToolUse', ({ result }) => ({
  replace: String(result).replace(/sk-or-\S+/g, 'sk-or-[redacted]'),
}));

// 3. Failures get a lifecycle too.
hooks.on('SessionEnd', ({ ok, reason, durationMs }) => ({
  note: `run ${ok ? 'ok' : `failed (${reason})`} in ${durationMs}ms`,
}));

printHookCatalogue({ hooks });
await runTask({ task, workspace, commandPolicy, maxSteps, chatFn, hooks });
```

With that attached, a `git push origin main` never runs, and the model is told so as a tool result:

```
tool → run_command {"command":"git push origin main"}
result ← DENIED: a PreToolUse hook blocked this call (no pushes from the sandbox). Try another approach.
```

## Tools versus hooks

> **A tool is how the model acts on the world; a hook is how you act on the loop.**

Tools are the model's hands, described in the prompt; the model chooses them. Hooks are yours: not described to the model, no vote from it, and they run whether it likes it or not. That asymmetry is the safety story of a real harness.

## Checkpoint

**1. Why are there exactly nine hooks, and how were they chosen?**

A. They are the places the loop already pauses; a hook for something the loop does not do is documentation, not an extension point.

**2. Which four hooks can change the run, and what can each change?**

A. `SessionStart` (refuse start), `UserPromptSubmit` (rewrite/block task), `PreToolUse` (block call), `PostToolUse` (replace observation).

**3. Why does a hook that throws not stop the loop?**

A. One broken extension must not break the loop's invariants, especially the tool-call/result pairing.

**4. Why is a `PreToolUse` block returned as an observation rather than thrown?**

A. A refusal is a normal outcome the model can recover from; a throw would end the run.

**5. Why does the first block win?**

A. A later hook must not be able to unblock a refusal; that would be confusing and unsafe.

**6. Why is the hook catalogue generated from the same data the loop uses?**

A. Generated from the same data the loop fires, so docs and behaviour cannot drift.

---

# Phase D: The agent loop and polish

**Goal of the phase:** extract the conversation loop into a testable library, add an offline demo that proves the safety guards, and polish the CLI with sessions, history trimming, a banner, and exit codes.

**What you can do at the end:** run a multi-task session, run a complete scripted demo with no network, and explain the entire lifecycle from a single banner.

---

# Stage 12: The ReAct loop

## What this stage is really about

Stage 12 formalises the loop as a library: `runTask`. It is the chapter where "the loop is the agent" becomes literal code you can read in one sitting.

The extraction is deliberately late. By now you have built the loop by hand in `cli.js`, added tools to it, and wired hooks into it. You know what belongs there. Extracting earlier would produce a library nobody understands.

## The signature

```
export async function runTask({
  task,
  workspace,
  sandbox = null,
  commandPolicy,
  maxSteps,
  chatFn,
  confirm,
  hooks = null,
  memory = null,
  history = [],
  log = console.log,
  heartbeatMs = 5000,
}) { /* ... */ }
```

Return shapes:

```
{ ok: true,  answer, messages, steps }
{ ok: false, reason, messages }
```

Why an object rather than throwing? Because a task outcome is data the CLI can inspect and turn into an exit code. `request_failed` and `max_steps` are expected outcomes, not exceptional ones.

## The golden rule

`agent.js` is a library. It knows nothing about flags, colours, `.env`, or `process.exit`. The CLI owns all process concerns. Tests can run the loop with a fake model and no network.

This is not purity for its own sake. It is what makes the loop testable:

- pass a fake `chatFn`;
- pass a temp `workspace`;
- pass a fake `confirm`;
- pass `log: () => {}`;
- assert on the returned `messages` and `ok`.

No network, no API key, no terminal. The entire Stage 15 test suite depends on this boundary.

### The seven beats per step

For `step = 1..maxSteps`:

1. **Print a step heading.** `ui.step(step, maxSteps)`.
2. **Fire** `PreModel` with `{ step, maxSteps, messages, tools: TOOL_SCHEMAS }`.
3. **Start the heartbeat and call the model.** - On throw: stop the heartbeat, print `ui.error`, fire `SessionEnd`, return `{ ok: false, reason: 'request_failed', messages }`. - On success: stop the heartbeat.
4. **Extract the choice and message.** Fire `PostModel` with `{ step, message, finishReason }`.
5. **Truncation guard.** If `finish_reason === 'length'` and there are no tool calls: - append the assistant message; - append a `user` message: `"You were cut off by the length limit. Continue with smaller tool calls."`; - print a warning; - `continue` to the next step. Never dispatch truncated tool calls.
6. **Append** `sanitizeAssistant(message)`**.** If `content` is non-empty, print `ui.think(...)`.
7. **Dispatch tool calls or finish.** - **No calls:** print the answer, fire `Stop`, fire `SessionEnd`, return `{ ok: true, answer, messages, steps: step }`. - **Calls:** for each in order:
   1. read `name` and `rawArgs`;
   2. print `ui.tool(name, rawArgs)`;
   3. fire `PreToolUse`; if blocked, observation = `DENIED: a PreToolUse hook blocked this call (<reason>). Try another approach.`; otherwise `await runTool(name, rawArgs, ctx)`;
   4. print `ui.result(result)`;
   5. fire `PostToolUse`; if `replace !== null`, use the replacement;
   6. append exactly one `{ role: 'tool', tool_call_id: call.id, content: String(post.replace ?? result) }`.

After the loop exhausts `maxSteps`:

- print `ui.error('stopped: hit the <maxSteps>-step limit')`;
- fire `StepLimit`, then `SessionEnd`;
- return `{ ok: false, reason: 'max_steps', messages }`.

### The truncation guard is the interesting one

A cut-off reply often ends mid-JSON:

```
{"path": "hello.js", "cont
```

Dispatching that would produce `ERROR: could not parse arguments` for a tool call the model never finished, and the model would spend a step apologising for a bug it did not have.

Refusing to dispatch it is the difference between a harness that respects the model and one that argues with it. The corrective message is a `user` turn, not a system change: the model is told what happened and asked to continue smaller.

### Why the system prompt is rebuilt

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
```

Two details are the whole trick:

1. **The system message is rebuilt**, so a memory saved by the previous task is already in the prompt the next task sends.
2. **The history is filtered** to remove any old system message, so there is exactly one.

If you carried the old system message forward, memory would be stale and the prompt would slowly accumulate duplicates.

### The hook default

```
const hook = hooks ?? new Hooks();
```

Why not `if (hooks) await hooks.run(...)` at every call site? Because that would litter the loop with conditionals. A null-object default lets every call site call `hook.run(...)` unconditionally. This is a small pattern worth learning: **make the absent case a working object, not a special case.**

### Entry guard: `agent.js` is not the program

Running `node src/agent.js` should print:

```
agent.js is the loop, not the program.
Run the CLI instead:  node src/cli.js --help
```

and exit. `agent.js` must not import `cli.js`. The dependency arrow points one way.

### The heartbeat

```
export function heartbeat(log, everyMs = 5000) {
  const started = Date.now();
  const timer = setInterval(() => {
    const seconds = Math.round((Date.now() - started) / 1000);
    log(ui.waiting(`  … still waiting for the model (${seconds}s)`));
  }, everyMs);
  timer.unref?.();
  return { stop: () => clearInterval(timer) };
}
```

Why `unref()`? A timer that holds the event loop open would turn a clean exit into a program that hangs for five more seconds. `unref` says "this timer must not keep the process alive on its own".

Why arm it just before the call and stop it just after, including on the failure path? Because a leaked timer prints "still waiting" after the run is over, which is worse than no heartbeat.

## Checkpoint

**1. Why is `runTask` a library rather than part of `cli.js`?**

A. So it can be tested and read without the CLI; the dependency arrow points one way.

**2. Why are `request_failed` and `max_steps` returned rather than thrown?**

A. They are expected task outcomes the CLI turns into exit codes; throwing would lose the message array.

**3. Walk through the seven beats from memory. Which beat fires which hook?**

A. Step heading → `PreModel`; model call; `PostModel`; truncation guard; append/sanitise; `Stop` / `PreToolUse`; `PostToolUse`; `StepLimit` / `SessionEnd`.

**4. Why is a truncated tool call not dispatched?**

A. A truncated call is half a JSON object; dispatching it produces a confusing error for a call the model never finished.

**5. Why is the system prompt rebuilt for every task?**

A. Memory may have changed, and a carried system message would be stale or duplicated.

**6. What does `hooks ?? new Hooks()` buy the call sites?**

A. It makes the absent case a working object, so call sites need no conditionals.

**7. Why does the heartbeat call `unref()`?**

A. So the timer never keeps the process alive after the run is finished.

---

# Stage 13: Offline demo

## What this stage is really about

Stage 13 proves the harness without a network, a key, or a model. A **fake model** replays a fixed script, and every other seam is the real one: the real loop, the real dispatcher, the real sandbox, the real memory tools. The only thing missing is one POST.

This is the stage where you learn that a demo can be a test.

## The fake model

```
export function demoChat() {
  let i = 0;
  return async () => {
    const message = DEMO_SCRIPT[Math.min(i, DEMO_SCRIPT.length - 1)];
    i += 1;
    return {
      choices: [{
        finish_reason: message.tool_calls ? 'tool_calls' : 'stop',
        message,
      }],
    };
  };
}
```

It has the same return shape as OpenRouter. That is the whole of `--demo`: one ternary in the CLI chooses `demoChat()` instead of the HTTP client.

## Why this works as a test

Because the reply is data, the script can do something a live model will not do on cue: deliberately try to escape the sandbox. Steps 6–10 do exactly that, and `sandboxScorecard()` reads the results back and prints which guards actually held.

That is a claim the program checks, not a slide.

## The script

The 14 tool steps plus a final answer:

1. `list_files` `{ path: '.' }`
2. `create_file` `{ path: 'demo/hello.js', content: ... }`
3. `run_command` `{ command: 'node demo/hello.js' }`
4. `edit_file` `{ path: 'demo/hello.js', old_string: ..., new_string: ... }`
5. `run_command` `{ command: 'node demo/hello.js' }`
6. `write_file` `{ path: '../escape.txt', content: 'must never be written' }` → expect `FORBIDDEN`
7. `read_file` `{ path: '../package.json', limit: 5 }` → allowed
8. `write_file` `{ path: '~/Desktop/pwned.txt', content: ... }` → expect `FORBIDDEN`

9. `run_command` `{ command: 'copy demo/hello.js %USERPROFILE%\\Desktop\\hello-copy.js' }` → expect `FORBIDDEN`
10. `run_command` `{ command: 'git push --force origin main' }` → expect `DENIED`
11. `run_command` that echoes `OPENROUTER_API_KEY` → expect `absent`
12. `run_command` that exits `1` → expect `exit code 1`
13. `remember` `{ text: ..., tag: 'fact' }`
14. `recall` `{ query: 'demo' }`

Then a final answer summarising everything.

Why these steps?

- 1–5 are the loop: look, create, run, edit, run.
- 6–10 are the fence: path escape, open read, home write, shell copy, force push.
- 11–12 are process safety: environment isolation, exit codes.
- 13–14 are persistence: write and read memory.

Every capability in the course appears at least once.

## The scorecard

`sandboxScorecard(messages)` maps `tool_call_id` to the tool result, runs regex checks, and returns rows:

```
[
  { ok: true,  label: 'a write that escapes the workspace is refused', seen: 'FORBIDDEN: SAVE ../escape.txt' },
  { ok: true,  label: 'reads are open - one folder up works',          seen: '../package.json (22 lines)' },
  { ok: true,  label: 'a home-folder write is refused',                seen: 'FORBIDDEN: SAVE ~/Desktop/pwned.txt' },
  { ok: true,  label: 'a shell copy out of the sandbox is refused',    seen: 'FORBIDDEN: COMMAND COPY ' },
  { ok: true,  label: 'dangerous commands are denied, whatever the policy', seen: 'DENIED: blocked by policy' },
  { ok: true,  label: 'the child process cannot see the API key',      seen: 'OPENROUTER_API_KEY=absent' },
  { ok: true,  label: 'run_command reports the real exit code',        seen: 'exit code 1' },
]
```

Why a scorecard rather than a few `console.log` lines?

- It is **checkable**: the run exits non-zero if any guard regressed.
- It is **readable**: each row says what was tested and what was seen.
- It is **teachable**: the audience sees the claim and the evidence side by side.

The demo exits `0` on 7/7 checks; otherwise `process.exitCode = 1`.

## Why `--demo` sets `maxSteps = DEMO_SCRIPT.length + 1`

The script has 14 tool steps, then a final answer. The loop needs 15 steps to reach the answer. Setting `maxSteps` to `15` makes the demo deterministic and prevents a bug in the script from turning into a step-limit failure. The `+ 1` is for the answer.

### The demo is also a re-run lesson

Run the demo a second time and step 2 fails: `demo/hello.js` already exists, so `create_file` refuses to clobber it. `edit_file` may also fail because the line has already been edited. Both arrive as strings, the loop absorbs them, and the run still ends with an answer and a clean scorecard.

That is not a bug in the demo. It is principle 4 in action, and exactly what a careful human would do.

## Checkpoint

**1. What does `demoChat()` replace, and what does it leave real?**

A. It replaces the HTTP client only; the loop, tools, sandbox, hooks and memory are real.

**2. Why can the demo test guards that a live model might not exercise?**

A. Because the script can deliberately attempt escapes that a live model might not try on cue.

**3. Why does `--demo` need a `maxSteps` of 15?**

A. 14 tool steps plus the final answer.

**4. Why is the scorecard better than a transcript claim?**

A. It is checkable: the process exits non-zero if a guard regresses, and each row shows the evidence.

**5. What happens on a second demo run, and why is that instructive?**

A. `create_file` refuses to overwrite, so the run demonstrates recoverable errors and still completes.

---

# Stage 14: Sessions, trimming and CLI polish

## What this stage is really about

Stage 14 is about the difference between a **task** and a **session**, and about making the program honest at the boundaries: history is trimmed, the banner is reproducible, and exit codes mean something.

## The session loop

```
let history = [];
while (true) {
  const line = await askTask();
  if (line === null || line === '/quit' || line === '/exit') break;
  if (line === '/new') { history = []; continue; }
  if (!line) continue;

  const result = await runTask({ ...base, task: line, history });
  history = trimHistory(result.messages);
  printSummary(result);
}
```

Three behaviours come straight out of the design:

1. **The step budget restarts.** `runTask` is one task, and its `for` starts at 1, so the second task gets the full 12 steps — not "step 5 of 12" continued from the first.
2. **The conversation is carried, the system prompt is not.** The CLI passes the last 24 messages as `history`, so "now add tests" knows what was just built; the system message is rebuilt, so a memory saved by task one is already in front of the model in task two.
3. `/new` **forgets the conversation, not the project.** Files stay, `src/memory.md` stays.

## `trimHistory`: why the cut must land on a `user` message

```
export function trimHistory(messages, max = MAX_HISTORY) {
  const kept = (messages ?? []).filter((m) => m.role !== 'system');
  let start = Math.max(0, kept.length - Math.max(0, max));
  while (start < kept.length && kept[start].role !== 'user') start += 1;
  return start >= kept.length ? [] : kept.slice(start);
}
```

Why not simply take the last 24 messages?

Because providers reject conversations that:

- start with a `tool` result (there is no assistant call that it answers), or
- end with an unanswered assistant `tool_calls` (the tool results were cut off).

Moving `start` forward until it lands on a `user` message guarantees a valid conversation boundary. If no safe boundary exists, return `[]` — a fresh conversation is better than an invalid one.

This is a subtle but important lesson: **truncation is not just about size; it is about protocol validity.**

## The banner

Before step 1, print a reproducible header:

```
min-coding-js
workspace : <root>
sandbox   : save, edit, delete - only inside <root> (reads: anywhere)
model     : <model> (from <source>) | scripted (no network)
key       : <fingerprint> from <source> | - in demo
env file  : <path> | <path> (missing)
commands  : allow | ask | deny
timeout   : <seconds>s per model call
tools     : <comma-separated TOOL_NAMES>
memory    : <summary> | off (--no-memory)
hooks     : on: each of 9 prints as [HOOK:name] ... | 9 available, not printed (--no-hooks)
```

Why this is worth the code:

- **It is one header, not scattered logs.** When a demo goes wrong, you read one block.
- **It records provenance.** `(from .env)` answers the first debugging question.
- **It records posture.** `commands: ask` tells you whether a pause is expected.
- **It records capability.** The tool list tells you what the model can do in this run.

The banner is a contract: what you see is what the run uses.

## Exit codes

| CODE | MEANING |
|---|---|
| 0 | The model stopped calling tools and gave an answer, or the session ended normally. |
| 1 | The request failed — bad key, no credits, timeout, non-JSON, fatal error. |
| 2 | No task: nothing typed at the prompt, the field closed, or stdin is not a terminal. |
| 3 | Stopped at the step limit. |

`taskExitCode(result)`:

```
if (!result.ok && result.reason === 'request_failed') return 1;
if (!result.ok && result.reason === 'max_steps') return 3;
return 0;
```

Why distinguish `1` and `3`? Because they mean different things to a caller:

- `1` is a configuration or transport problem; retrying without a change is pointless.
- `3` is an autonomy problem; the task may need a tighter prompt or a larger budget.

A script can branch on the difference.

## The entry guard

```
if (import.meta.url === pathToFileURL(process.argv[1]).href) {
  main().catch((err) => {
    console.error(`fatal: ${err.stack}`);
    process.exitCode = 1;
  });
}
```

Why check that `cli.js` is the entry file? So that importing `cli.js` from a test does not run the program. This is the same lesson as `agent.js` being a library: **module top-level code should not have side effects.**

## Warning on a non-OpenRouter model

If live mode is on and the model lacks a `/`, the banner warns:

```
warning: model is not an OpenRouter slug...
```

If normalisation changed a bare name, a dim note shows the final slug. A helpful guess that announces itself is a convenience; a silent one is a future bug.

## The session summary

After each task:

```
── task finished ──────────────────────────────────────
  ✓ answer in 4 step(s) - 9 message(s) carried into the next task
  type another task, /new to clear the conversation, /quit to exit
```

Why print the message count? Because it makes the cost of history visible. The next task sends that many messages plus the system prompt. When the count grows, the bill grows.

## Checkpoint

**1. What is the difference between a task and a session?**

A. A task is one `runTask` call; a session is multiple tasks in one process carrying history.

**2. Why does the step budget restart for each task?**

A. The step budget lives in `runTask`'s `for`, which starts at 1 per task.

**3. Why must `trimHistory` start on a `user` message?**

A. Providers reject histories starting with a tool result or ending with an unanswered tool call.

**4. What does the banner record, and why is provenance part of it?**

A. Workspace, sandbox posture, model and provenance, key fingerprint, env file, policy, timeout, tools, memory, hooks.

**5. What do exit codes `1`, `2` and `3` each mean?**

A. `1` request/fatal failure; `2` no task/no TTY; `3` step limit.

**6. Why is `main()` called only when `cli.js` is the entry file?**

A. So importing `cli.js` from a test does not run the program.

---

# Phase E: Hardening and documentation

**Goal of the phase:** prove the harness offline, and write documentation that teaches the architecture rather than just the commands.

---

# Stage 15: The full test suite

## What this stage is really about

Stage 15 is where the library boundary pays off. Every seam is injectable, so the whole harness can be tested with:

- a fake `chatFn`;
- a temporary directory;
- an injected `log`;
- a fake `confirm`;
- no network, no key, no terminal.

## The shape of a good test suite

The suites map to the architecture:

| SUITE | WHAT IT PROVES |
|---|---|
| `workspace.test.js` | Path policy, containment, symlink safety. |
| `sandbox.test.js` | Refusals, root deletion, tool-layer conversion. |
| `tools.test.js` | All eight tools, validation, `safeEnv`, policy. |
| `shell-sandbox.test.js` | Tokenisation, write-target scanning. |
| `agent.test.js` | Loop behaviour, truncation, heartbeat, demo guards. |
| `hooks.test.js` | Registry, blocking, replacing, tracing, catalogue. |
| `memory.test.js` | Round-trip, duplicates, bounds, digest, forget, search. |
| `prompt.test.js` | Rules, injection, logging. |
| `llm.test.js` | Timeout, errors, sanitisation. |
| `config.test.js` | `.env` precedence, fingerprint, model normalisation. |
| `session.test.js` | Commands, trimming, follow-up tasks. |
| `ui.test.js` | Styling, verbatim output, result classification. |
| `input.test.js` | `askTask`, blank lines, Ctrl-C. |

Why this mapping matters: a test suite that mirrors the module graph is a test suite you can navigate. When `sandbox.js` changes, you know exactly which tests should fail.

## Testing the model client without a network

Mock `globalThis.fetch`:

```
const original = globalThis.fetch;
globalThis.fetch = async (url, init) => {
  assert.match(init.body, /"model":"deepseek/);
  return new Response(JSON.stringify({
    choices: [{ message: { role: 'assistant', content: 'hi' } }],
  }), { status: 200, headers: { 'Content-Type': 'application/json' } });
};
try {
  const reply = await chat({ apiKey: 'test', model: 'deepseek/deepseek-v4.1-flash-1017', messages: [] });
  assert.equal(reply.choices[0].message.content, 'hi');
} finally {
  globalThis.fetch = original;
}
```

Always restore the original in a `finally`. A leaked mock makes the next test lie.

Test the failure paths too:

- `401` → error mentions bad key;
- `402` → error mentions credits;
- non-JSON body → error includes a truncated preview;
- timeout → error says `timed out`;
- missing key → error says `OPENROUTER_API_KEY is not set`.

## Testing the loop without a model

```
const chatFn = async ({ messages }) => ({
  choices: [{
    finish_reason: 'stop',
    message: { role: 'assistant', content: 'done' },
  }],
});

const result = await runTask({
  task: 'do nothing',
  workspace,
  commandPolicy: 'deny',
  maxSteps: 3,
  chatFn,
  confirm: async () => 'deny',
  log: () => {},
});

assert.equal(result.ok, true);
assert.equal(result.answer, 'done');
assert.equal(result.steps, 1);
```

For a tool call, return a `tool_calls` message, then a final answer on the next call. Use a counter in the fake model.

## Testing the truncation guard

Return `finish_reason: 'length'` with no tool calls, then a normal answer. Assert:

- the truncated message is appended;
- a `user` corrective message is appended;

- the tool call is not dispatched;
- the loop continues.

### Testing hooks

- Attach a hook and assert the payload shape.
- Attach two and assert order.
- Return `{ block: true }` and assert the next hook does not run.
- Throw from a hook and assert the loop continues and the error is collected.
- Attach `{ once: true }` and assert it runs once.
- `off` by id and assert it is gone.
- Unknown event name → throws.

### Testing memory with a temp file

Use `fs.mkdtempSync(path.join(os.tmpdir(), 'memory-test-'))`, point `Memory` at a file inside it, and clean up in `after`. Assert:

- write and read back survive a new `Memory` instance;
- duplicate refreshes the date, no duplicate bullet;
- `maxEntries` drops the oldest;
- long text is truncated;
- empty text is refused;
- digest is bounded and states the hidden count;
- `forget` removes matches;
- hand edits are picked up immediately.

### Why `npm test` must be offline

Because a test suite that needs a key is a test suite nobody runs. The offline suite is the contract: **the whole harness can be exercised without a network.** That is only possible because of the library boundary from Stage 12 and the fake model from Stage 13.

### Checkpoint

**1. Why does the test suite mirror the module graph?**

A. It makes failures easy to locate: a module change maps to a suite.

**2. How do you mock `fetch` safely?**

A. Replace `globalThis.fetch`, assert on the request, restore in `finally`.

**3. Why must the loop be testable without a model?**

A. Because the loop is the agent; testing it without a network is the proof of the library boundary.

**4. What are the three most important loop tests?**

A. A plain answer stops; tool calls are dispatched and fed back; a failure result is fed back and recovered from. Add truncation and step-limit tests.

**5. Why must memory tests use a temporary directory?**

A. Memory writes to disk; a real `src/memory.md` would make tests order-dependent and destructive.

---

# Stage 16: Documentation and final acceptance

### What this stage is really about

Documentation is part of the architecture, not an afterthought. A README that only lists commands teaches nothing; a README that explains the loop, the tools, the safety model and the extension points teaches the reader to maintain the harness after you leave.

### What the README must contain

- A one-paragraph summary.
- Quick start: `npm install`, copy `.env.example` to `.env`, set the key, `npm start`, `npm run demo`.
- The loop diagram and the seven beats.
- A file-by-file tour, with `agent.js` first.
- The eight tools and what each is for.
- The read/write asymmetry and the shell limitation.
- The nine lifecycle hooks, with a table.
- Long-term memory: file format, eager digest vs lazy recall, `--memory`.
- Sessions and history trimming.
- A configuration table with defaults and precedence.
- Exit codes.
- The safety model: workspace fence, hard denies, environment allow-list, command policy, and an explicit "this is not a container" statement.
- Examples: a Desktop write that ends in `FORBIDDEN`, a prompt that ends with `remember`.

### Why "this is not a container" belongs in the docs

Because overclaiming safety is worse than admitting limits. A reader who believes the sandbox is isolation may run the agent on a machine where a mistake matters. A reader who understands it is a path policy and a command-line seatbelt will make an informed choice.

The same honesty applies to the other limits:

- no streaming;
- no context compaction, just a cut;
- no sub-agents, retries or skills;
- memory is one markdown file with substring search;
- one tool at a time in spirit.

Each limit is where a production agent grows a layer. Naming them tells the reader where to extend.

`demo_prompt.md`

Provide copy-pasteable prompts that exercise the whole system:

- `create hello.js that prints today's date and time, run it, and save its output to hello-output.txt`;
- `save a file test.txt that contains 'HELLO WORLD' and save to the desktop` (must end in `FORBIDDEN`);
- `npm run demo`, `node src/cli.js --memory`, `node src/cli.js --demo`;
- `work out how this package is tested, run the tests, and remember the command...`.

Why a prompt file? Because a demo is a performance, and a performance needs a script. It also doubles as an acceptance test: if a prompt stops producing the advertised result, the documentation is stale.

## Final acceptance

The twenty checks in `PLAN.md` are the contract. Group them mentally:

- **Install and run:** `npm install`, `npm test`, `--help`, `--hooks`, `--memory`.
- **Demo:** `--demo` completes 14 steps, 7/7 checks, exit 0.
- **Live mode:** with a valid key, a task runs; without a key, exit 1.
- **Input edge cases:** no task + no TTY exits 2; step limit exits 3.
- **Safety:** write outside workspace refused; read outside allowed; `CMD_POLICY=deny` refuses commands; hard denies survive `--yes`; child cannot read the key.
- **Persistence and observability:** memory survives restarts; every run prints hook traces; `--no-hooks` silences them.
- **Sessions:** `/new`, `/quit`, `/exit`; follow-up sees history but a fresh prompt.
- **Architecture:** `agent.js` direct execution prints a hint; no framework, no agent SDK, no database, no hidden global state.

If all twenty hold, you have not just built a demo. You have built a harness whose behaviour you can explain, test and extend.

## Checkpoint

**1. Why is the README part of the architecture?**

A. It teaches the architecture and the limits; without it, the harness is a black box that only you can maintain.

**2. Why must the docs state the sandbox's limits?**

A. Overclaiming safety leads to unsafe use; stating limits lets the reader make an informed choice.

**3. What makes a good demo prompt?**

A. One that exercises a real capability, has an observable result, and optionally demonstrates a guard failing safely.

**4. Pick three final acceptance checks and explain which design decision makes each one pass.**

A. Example: "write outside workspace refused" passes because `Workspace.resolve` is strict and the sandbox converts `PathError` into `FORBIDDEN`; "child cannot see key" passes because `safeEnv` allow-lists variables; "step limit exits 3" passes because `runTask` returns `reason: 'max_steps'` and `taskExitCode` maps it.

---

# Appendices

# A. The message protocol

The agent's entire state is an array of messages. Four roles:

| ROLE | WRITTEN BY | REQUIRED FIELDS | NOTES |
|---|---|---|---|
| `system` | harness | `content` | Rebuilt per task; exactly one. |
| `user` | harness | `content` | The task or a corrective nudge. |
| `assistant` | model | `content`, optional `tool_calls` | Sanitised before appending. |
| `tool` | harness | `tool_call_id`, `content` | One per tool call; content is always a string. |

Rules:

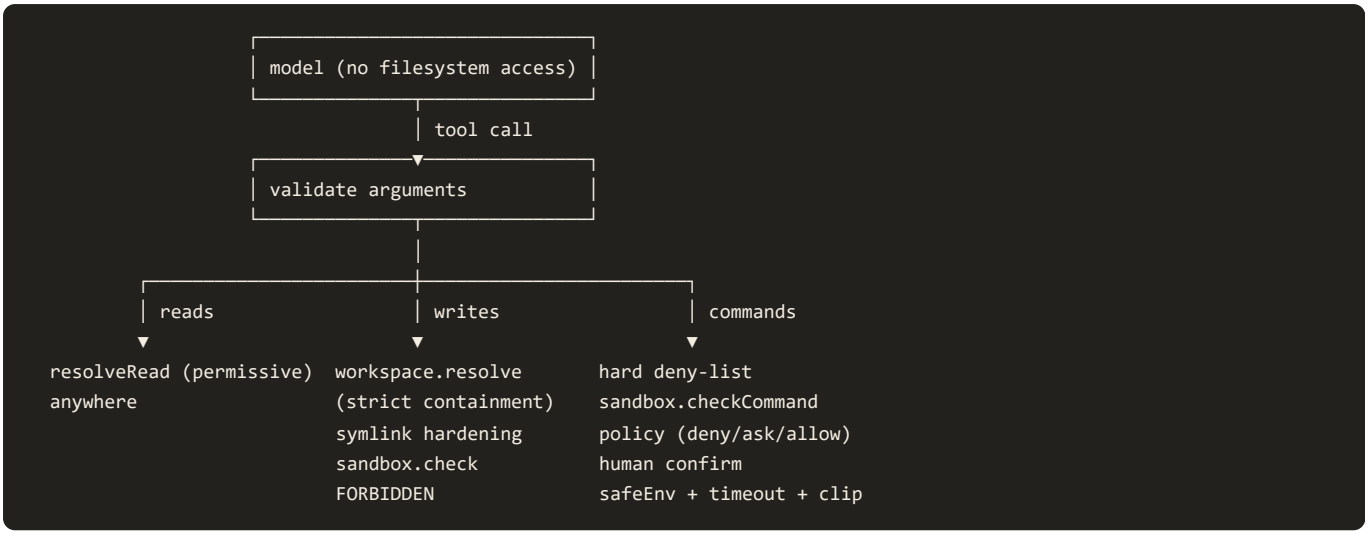
1. **Every `tool_call_id` gets exactly one matching `tool` message.** Otherwise the next request is malformed.
2. **Tool content is a string, even for errors.** `ERROR: ...`, `DENIED: ...`, `FORBIDDEN: ...`.
3. **The assistant message is appended before its tool results.** Order matters.
4. **The system message is not carried in history.** Filter it out and rebuild.
5. **A truncated assistant reply with no tool calls is appended, then followed by a `user` corrective message.** Never dispatch half a call.

## A tool call in the wire format

```
{
  "id": "call_abc",
  "type": "function",
  "function": {
    "name": "read_file",
    "arguments": "{\"path\":\"src/cli.js\",\"limit\":40}"
  }
}
```

`arguments` is a JSON **string**. You must parse it. A model that emits invalid JSON gets an `ERROR: ... arguments are not valid JSON` observation, not a crash.

# B. The safety model in one page




The five layers:

1. **Workspace containment** — writes must resolve inside `workspace/`, including after variable expansion and symlink resolution.
2. **Sandbox gate** — one place prints `FORBIDDEN` and converts policy errors into observations.
3. **Command scanning** — visible shell write targets are checked with the same policy before the command runs.
4. **Command policy** — `deny`, `ask`, `allow`; `ask` degrades to deny without a TTY.
5. **Process hygiene** — minimal child environment, timeout, output cap.

What it is not:

- not a container;
- not a syscall filter;
- not protection against a script that writes files;
- not a substitute for running the agent somewhere disposable.

# C. Every bound and cap

| BOUND | DEFAULT | WHERE | WHAT IT PREVENTS |
|---|---|---|---|
| `maxSteps` | 12 | `config.js`, `runTask` | A model that never concludes. |
| Request timeout | 90 s | `llm.js` | A socket that never answers. |
| `max_tokens` | 2048 | `llm.js` | Whole files in one reply; runaway cost. |
| Command timeout | 30 s | `tools.js` | A command that hangs the run. |
| Command output cap | 6,000 chars/stream | `tools.js` | A chatty command eating the context window. |
| `maxBuffer` | 1 MB | `child_process` | Memory exhaustion from output. |
| `list_files` depth | 3 (max 6) | `tools.js` | Recursive listings that explode. |
| `list_files` lines | 200 | `tools.js` | A giant listing in context. |
| `read_file` limit | 200 (max 2,000) | `tools.js` | Reading a huge file in one step. |
| Memory entries | 60 | `memory.js` | The file growing without limit. |
| Memory prompt chars | 1,200 | `memory.js` | The digest eating the context window. |
| Memory entry chars | 300 | `memory.js` | One note dominating. |
| `remember` chars | 2,000 | `memory.js` | One call dominating. |
| Digest entries | 12 | `memory.js` | Too many memories in the prompt. |
| `MAX_HISTORY` | 24 | `cli.js` | A session growing without bound. |
| `maxEntries` / tool caps | — | tests | Regression of any of the above. |

Every bound has a visible consequence rather than a silent stall. That is the design rule.

# D. Debugging by symptom

| SYMPTOM | FIRST PLACE TO LOOK |
|---|---|
| `401` / bad key | `.env` precedence, `fingerprint`, `--model` flag. |
| `402` | OpenRouter credits, not the harness. |
| Model never calls tools | Tool schemas not passed, prompt does not mention tools, model lacks function calling. |
| Tool call arguments fail to parse | Truncation (`finish_reason: length`), `max_tokens`, model emits prose. |
| `old_string not found` | Model hallucinated whitespace; read the file first; copy exactly. |
| `old_string appears N times` | Add context or `replace_all: true`. |
| `FORBIDDEN` on a legitimate path | Path policy; check the first segment, `~`, `%VAR%`, symlinks. |
| Command never runs under `ask` | No TTY → deny by design. |
| Command runs without asking | `--yes`, `CMD_POLICY=allow`. |
| Child cannot find a command | `safeEnv` dropped a needed variable; check the allow-list. |
| Model loops forever | `maxSteps`; look for a tool result it cannot interpret. |
| Memory not injected | `--no-memory`, `MEMORY_PROMPT_CHARS=0`, empty digest. |
| Memory grows without bound | `MEMORY_MAX_ENTRIES`, duplicate handling. |
| Hooks do not fire | `--no-hooks`, event name typo, handler attached to the wrong event. |
| Hook blocks everything | Payload inspection; first block wins; check the reason. |
| Session forgets the previous task | `trimHistory`, `/new`, history cut on a tool message. |
| Provider rejects history | Cut lands on a `tool` result or an unanswered `tool_calls`. |
| Program hangs after a run | Heartbeat timer not `unref`'d or not stopped. |

# E. Glossary

**Agent** — a model plus tools plus a loop. The loop is the distinguishing part.

**Assistant message** — the model's reply: `content` and possibly `tool_calls`.

**Bounded autonomy** — every loop and every expensive operation has a cap.

**Digest** — the bounded, newest-first memory block injected into the system prompt.

**Dispatcher** — `runTool`: validates arguments, finds the handler, catches errors, returns a string.

**Eager memory** — the digest read at the start of a run.

`edit_file` — exact-snippet replacement with ambiguity detection.

`ForbiddenError` — typed refusal from the sandbox, carrying action, requested path and reason.

**Hard deny-list** — command patterns refused before policy is consulted.

**Heartbeat** — a timer that prints "still waiting" during a slow model call.

**Hook** — a function attached to a named lifecycle moment; may observe, rewrite or block.

**Lazy memory** — `recall`, a search performed on demand.

`maxSteps` — the number of model calls a single task may make.

**Message array** — the agent's entire working state.

**Path policy** — the rules in `workspace.js` that decide whether a path is contained.

**Provenance** — where a configuration value came from (flag, `.env`, shell, default).

**ReAct** — Reason, Act, Observe: the loop pattern.

`runTask` — the library function that runs one task.

`safeEnv` — the child-process environment allow-list.

**Sandbox** — the write fence around save/edit/delete and shell writes.

**Sanitise** — rebuild an assistant message with only the fields the harness needs.

**Schema** — the JSON description of a tool that the model reads.

**Session** — multiple tasks in one process, carrying conversation history.

**System prompt** — the model's job description, rebuilt per task.

**Tool** — a named capability with a schema and a handler.

`tool_call_id` — the identifier that ties a tool result to the assistant's request.

**Truncation guard** — refusing to dispatch a reply cut off by the length limit.

**Workspace** — the folder the agent may write to.

# F. Reading the finished code

If you want to read the finished harness in the order that teaches the most:

1. `agent.js` — the whole loop, 215 lines including comments. Read it twice.
2. `prompt.js` — what the model is told, including memory.
3. `tools.js` — the schemas *are* the prompt; read the descriptions carefully.
4. `workspace.js` and `sandbox.js` — the safety story, in that order.
5. `hooks.js` — the extension points.
6. `memory.js` — the persistence story.
7. `llm.js` — one POST, plus sanitisation.
8. `config.js` — precedence and provenance.
9. `cli.js` — the program, if you need it.
10. `demo.js` — what runs when the wifi does not.

Read `agent.js` first and in full. It is the spine. Everything else is a dependency of it or a caller of it.

---

# Closing note

You now have the full conceptual ladder:

- a model call is just a POST;
- a conversation is just an array;
- a tool is a schema plus a function that returns a string;
- a sandbox is a policy, not a container;
- a hook is a named pause;
- memory is a file;
- the loop is the agent.

Every one of those sentences is small enough to hold in your head, and together they explain the behaviour of every coding agent you will meet. The rest is layers.